FedECS-LT: Exact Class Statistics for Long-Tailed Federated Learning with Heterogeneous Client Models

[First Author][a,*], [Second Author][a]

[a] *[Department, Institute], [City], [Postcode], [Country]*

Abstract

Federated learning across clients that run different model architectures and hold long-tailed, skewed label distributions loses most of its accuracy on rare classes. We study generator-based model-heterogeneous federated learning, where clients share a frozen extractor and a federated conditional feature generator, and each architecture trains its own head. We trace the loss to three places: Adam with coupled weight decay shrinks the generator's class-specific parameters on clients that lack a class; each head's last layer is fitted to one client's label prior; and synthetic features that do not carry their label penalise the spread of the logits. We propose FedECS-LT, which replaces each of these estimates by an exact class statistic computed under secure aggregation: a mean-anchored generator, class-count weighted averaging when classes are poorly separated, moment correction of generated features, and logit-adjusted heads trained on consensus soft labels; a class-balanced readout computed from aggregated sums serves to compare the bodies. With ten client architectures, the heads of FedECS-LT as trained reach 42.8 ± 2.0 balanced accuracy on CIFAR-10-LT and 14.3 ± 0.6 on CIFAR-100-LT, $+5.2$ and $+4.3$ points above the strongest of eleven other methods (paired tests: significant on CIFAR-100-LT, not on CIFAR-10-LT); when every method of the GeFL-F pipeline is given the same exact readout, the margins shrink to $+1.4$ and $+1.5$ points and are not significant. It also has the highest mean balanced accuracy on all six datasets with natural class imbalance (significant on two). With one model shared by all clients, the strongest long-tail methods are better than FedECS-LT. All methods share the same client splits.

*Corresponding author

Email address: [email] ([First Author])

Keywords: federated learning, model heterogeneity, long-tailed learning, class imbalance, synthetic features, secure aggregation

1. Introduction

Federated learning (FL) trains a model on data that stay with their owners [1, 2]. Two properties of real deployments make it hard. The participants run different networks on different hardware, so their parameters cannot be averaged. Their label distributions differ and follow a long tail, so the rarest classes are held by few participants and in small numbers. Each property has its own line of work: model-heterogeneous FL [3, 4, 5, 6, 7, 8] lets clients with different architectures share knowledge through prototypes, logits, generators or shared sub-networks, and federated long-tailed learning [9, 10, 11, 12, 13] corrects the bias that a skewed, long-tailed label distribution induces in a single shared model. Little is known about the two together, although most deployments that have one also have the other.

We study generator-based model-heterogeneous FL, in the form of GeFL-F [8]. Its clients first train a small feature extractor by federated averaging (FedAvg) and freeze it; they then train a conditional feature generator by FedAvg; finally every client trains a head of its own architecture on generated features and on its own real features. The generator is the only channel through which an architecture learns about classes its clients lack. GeFL-F was evaluated on balanced clients. Under long-tailed label skew we find that it loses most of its accuracy on the rare classes, and we ask where.

Where the rare classes are lost.. Because a client partition is a pure label shift, federation cannot bias class-conditional quantities by itself (Section 3); it can lose rare classes only through the optimiser, through parameters tied to a client's label prior, and through sample size. We locate three such losses (Section 4).

1. *The generator.* With Adam [14] and coupled weight decay, a client that lacks a class moves the generator's class-specific conditioning parameters toward zero at Adam's full step size, while the clients that hold the class barely grow them. Flat averaging then shrinks the rare classes' parameters round after round.
2. *The last layer.* Each head is trained on one client's real data, so its last layer fits that client's label prior. Per-client logit corrections

(logit adjustment [15], FedLC [16], Balanced Softmax [17]) are exact reparametrisations of such a head: training with them is plain training from a shifted initial bias. FedRS [18] is one up to a smaller step size on the absent classes' rows. None of them brings in information about the classes the client lacks.

3. *The synthetic data.* When generated features do not carry their label, the synthetic phase of head training penalises the spread of the logits instead of teaching the missing classes.

Our method.. FedECS-LT (Exact Class Statistics for Long-Tailed FL) replaces each estimate that fails by a class statistic that secure aggregation [19, 20] computes exactly: the per-class counts, sums and squared norms of the features over all clients. The generator is anchored to the exact class means and has no class-specific parameter for the decay to shrink; when the classes are too poorly separated for an anchor, a gate computed from the same statistics falls back to class-specific parameters averaged over their holders only (class-count weighted averaging). Generated features are rescaled to the exact class means and spreads. Heads are trained with logit adjustment and with consensus soft labels, the averaged prediction of all architectures on each generated feature. After training, every architecture's last layer is replaced by a class-balanced linear readout computed from aggregated sums of its embeddings. None of these statistics is new on its own: aggregated class statistics underlie classifier calibration [21], closed-form federated classifiers [22, 23, 24], including an LDA-type classifier from class statistics [24], and prototype methods [3], and statistics-conditioned generators have been proposed for heterogeneous clients [25]. What we add is the diagnosis of *which* parameters fail under a long tail and the use of exact statistics at exactly those places, including a readout per architecture that needs statistics pooled across architectures.

Evaluation.. We re-implemented the pipeline and every comparison of Section 7 in the Flower framework [26], with the published methods taken from their authors' or library code (HtFLlib [27], LongTailLib/FedLTLib [28]). We evaluate in two protocols: the model-heterogeneous setting of GeFL-F (ten client architectures; long-tailed CIFAR-10 and CIFAR-100 and, with natural class imbalance, DermaMNIST, OrganAMNIST, BloodMNIST, PathMNIST, EMNIST (ByClass) and FER-2013; eleven other methods) and the standard federated long-tailed protocol with one shared ResNet-8 (CIFAR-10 and CIFAR-100-LT at imbalance ratio 100; eleven other methods). Every

result is a mean over seeds 3–5 (fewer where a table says so), with paired tests and a Holm correction, and every metric is logged per class.

Contributions..

1. A long-tail evaluation of generator-based model-heterogeneous FL and a stage-by-stage account of where it loses rare classes, with short exact results checked against measurements (Section 4).
2. FedECS-LT, which uses exact, securely aggregated class statistics in the generator, the heads and the readout (Section 5).
3. A comparison against twenty-one published methods on eight datasets in two protocols, run on the same client splits, with paired statistics, per-class metrics and the cost of every method (Section 7). With different client models, FedECS-LT has the highest balanced accuracy on both long-tailed benchmarks and all six naturally imbalanced datasets; most of its margin comes from the last layer, and with the same exact readout for every method of the GeFL-F pipeline it remains +1.4 to +1.5 points ahead on the CIFAR benchmarks, a difference that is not significant with three seeds. With one shared model, the strongest long-tail methods are better than FedECS-LT, and we report which and by how much.

The code, every log and every figure's data are released (Section 9).

2. Related work

Model-heterogeneous federated learning.. When clients run different architectures, their parameters cannot be averaged and knowledge is exchanged in another form. Distillation methods exchange predictions on public or synthetic inputs (FedMD [29], FedDF [30]). Prototype methods exchange class means of embeddings (FedProto [3]) or learn global prototypes on the server (FedTGP [4]). Other methods share a part of the network: the head (LG-FedAvg [31], FedGH [6]) or a small homogeneous model next to each client's own (FedMRL [7]). Generator-based methods train a generator on the server against the ensemble of client predictors (FedGen [5]; FedFTG [32] does so for one shared model, in input space), let each client train a decoder whose latent prior is centred on server-averaged class prototypes (FedVTC [25], which generates images for generalisation to unseen data rather than for long-tailed classes), or train a conditional generator on the clients by FedAvg (GeFL,

GeFL-F [8]). HtFLlib [27] benchmarks six of these methods under label skew, with personalised evaluation on each client's test split and the best round, but not under a long-tailed global distribution. FedECS-LT keeps GeFL-F's three-stage design and changes how its class-specific quantities are estimated.

Federated learning under label skew and long tails.. With one shared model, the classifier is the most biased part under label skew. Classifier calibration re-trains it on virtual features drawn from aggregated class statistics (CCVR [21]) or on federated features learned on the server (CReFF [9]); closed-form classifiers are computed from aggregated sums of pre-trained features (Fed3R [22], a ridge-regression classifier; AFL [33], a least-squares head; FedNCM [23], nearest class means; FedCOF [24], an LDA-type classifier with a shrunk within-class covariance estimated from class means). These works also note that such sums do not depend on the partition. Our readout is the shared-covariance discriminant of the same family, with equal class priors, computed for each architecture on trained rather than pre-trained bodies; every client therefore runs every architecture's body on its own features; FedETF fixes the classifier to a simplex equiangular tight frame [34]. Loss corrections use each client's label frequencies (FedLC [16], FedRoD [35]) or the set of classes it holds (FedRS [18]), and long-tail methods re-balance gradients (Fed-GraB [10]), decouple a generic and a local head (FedLoGe [11]), combine prototype alignment with classifier rectification (RUCR [13]) or distil between augmented views of local data (FedYoYo [12]). FedLTLib [28] benchmarks most of them in one protocol, which we follow for the one-shared-model setting. Sampling features from Gaussians with the pooled class moments, as in CCVR and DFL-FS [36], is the closest prior to our moment correction, which instead maps a learned generator's samples onto the same moments. Class-specific weights are also known to degrade when clients lack classes (FedAwS [37], FedRS). These methods assume one architecture; their readout ideas apply to each of our architectures only if the statistics are pooled across architectures, which FedECS-LT does.

Long-tailed learning, optimisers and synthetic data.. FedECS-LT builds on logit adjustment [15], effective numbers of samples [38] and the decoupling of representation and classifier [39]. The generator analysis rests on the difference between coupled L2 regularisation and decoupled weight decay in Adam [40, 14]. Class-conditional generators can collapse when trained on little data, even centrally [41], and classifiers trained on generated data show

Table 1: Methods in this study. Different models: clients may run different architectures. Long tail: designed for long-tailed or skewed label distributions. Synthetic: trains or uses synthetic data or features. Extra data: needs data that no client holds (a public or proxy set). [a]FedDF needs public data; here it distils on GeFL-F's generated features.

Method	Different models	Long tail	Synthetic	Extra data
FedAvg [1], FedProx [45], MOON [46]	–	–	–	–
FedLC [16], FedRS [18], Balanced Softmax [17]	–	✓	–	–
CCVR [21], CReFF [9]	–	✓	✓	–
FedETF [34], Fed-GraB [10], FedLoGe [11]	–	✓	–	–
RUCR [13], FedYoYo [12]	–	✓	–	–
FedProto [3], FedTGP [4]	✓	–	–	–
FedALA [47] (within each architecture)	–	–	–	–
AvgKD [48]	✓	–	–	–
LG-FedAvg [31], FedGH [6], FedMRL [7]	✓	–	–	–
FedGen [5], GeFL-F [8]	✓	–	✓	–
FedDF [30]	✓	–	✓[a]	✓[a]
FedECS-LT (ours)	✓	✓	✓	–

how much class information the samples carry [42]. Weighting clients by their class counts when combining class-specific knowledge goes back to FedGen's ensemble weights [5]; the gate statistic is a trace form of Fisher's discriminant ratio [43]. Generators matched to kernel mean embeddings of the data have been used for private data release [44].

Table 1 places the methods we compare against.

3. Problem setting

Clients and data.. There are K clients and C classes. Client k holds a dataset D_k with n_{kc} samples of class c; $n_k = \sum_c n_{kc}$, $n_c = \sum_k n_{kc}$ and $N = \sum_c n_c$. The global class sizes follow a long tail, $n_c \propto \rho^{-c/(C-1)}$ with imbalance ratio $\rho = n_0/n_{C-1}$, and each class is divided among the clients by a Dirichlet draw with concentration α [49]. $H_c = \{k : n_{kc} > 0\}$ is the set of the $m_c = |H_c|$ clients that hold class c. Client k trains a model of architecture $a(k)$, one of G architectures. The goal is a model per architecture with high *balanced* accuracy, the mean recall over classes, on a test set in which every class is equally important.

The pipeline of GeFL-F.. GeFL-F [8] trains in three stages.

1. *Stage I.* A small feature extractor ϕ is trained together with the heads by FedAvg and then frozen. Every client maps its images to features $h = \phi(x)$.

2. *Stage II.* A conditional variational autoencoder [50] $G(h \mid y)$ is trained on the features by FedAvg, with Adam and coupled weight decay. The label enters through a linear projection whose c-th column, the *conditioning row* r_c, is the only class-specific parameter.
3. *Stage III.* In every round each client trains its head for T_s epochs on generated features with uniformly drawn labels and then for T_r epochs on its real features; heads of the same architecture are averaged.

A client split is a pure label shift.. Within a class, the split assigns samples to clients uniformly at random. So for every client k and class c with $n_{kc} > 0$, the client's class-c samples are a uniformly random subset of all class-c samples, and $P_k(x \mid y) = P(x \mid y)$; the same holds for the features $h = \phi(x)$, since ϕ is shared. Every client's data therefore point at the centralised target of any class-conditional quantity: a class mean, a class's conditioning row, a readout. Federation can make such a quantity noisier or slower to learn, but it can make it *biased* only through (i) the optimiser, (ii) parameters coupled to a client's label prior, such as biases and last layers, (iii) what the synthetic data teach, and (iv) sample size, which biases nonlinear estimates such as a spread or an inverse covariance in small classes. Section 4 locates each.

Secure aggregation.. Secure aggregation [19, 20] lets the server learn the sum of the clients' vectors and nothing else about any single vector. A statistic that is a sum over clients of per-class quantities is therefore computed exactly, and equals its value on the pooled data, whatever the partition. FedECS-LT uses only such statistics.

4. Where generator-based heterogeneous FL loses rare classes

This section locates the three losses that FedECS-LT repairs. Each comes with a short exact statement (proofs in Appendix A) and a measurement on the GeFL-F runs of Section 7.

4.1. The generator: coupled decay shrinks rare classes' conditioning rows

In one round, client k starts from the global row r_c, runs local Adam steps and returns $r_{k,c}$. Let $\Delta_k = \|r_{k,c}\| - \|r_c\|$, let $\bar{G}_c$ be the mean of Δ_k over the holders H_c and $\bar{D}_c$ the mean of $-\Delta_k$ over the other clients, and $p_c = m_c/K$.

Proposition 1 (contraction under flat averaging)**.** *Under* $r_c^+ = \frac{1}{K}\sum_k r_{k,c}$, $\|r_c^+\| \le \|r_c\| + p_c\bar{G}_c - (1 - p_c)\bar{D}_c$, *with equality if and only if all* $r_{k,c}$ *are*

non-negative multiples of one another. The row contracts whenever $p_c < \bar{D}_c/(\bar{G}_c + \bar{D}_c)$.

The bound only describes the measured changes; the optimiser decides how large $\bar{D}_c$ is.

Lemma 2 (coupled decay under Adam)**.** *On a client without class* c*, the gradient of the loss with coupled L2 regularisation* λ *with respect to* r_c *is exactly* λr_c*. Since Adam is invariant to a positive rescaling of the gradient sequence up to its* ϵ*, each step of such a client moves every coordinate of* r_c *toward zero by about the learning rate* η*, independently of* λ*, as long as* $\lambda|r_{c,j}| \gg \epsilon$*. With* $\lambda = 0$ *this shrinkage vanishes; with decoupled weight decay [40] it becomes a factor* $1 - \eta\lambda$ *per step, which is negligible here.*

A holder's data gradient fits the row and need not increase its norm, so $\bar{G}_c$ can be much smaller than $\bar{D}_c$, and rare classes' rows can contract even when most clients hold the class; a threshold on m_c alone cannot decide it. Counting Adam steps gives a predictor that uses only the class counts and the number of local steps q_k: with f_{kc} the probability that a batch of client k contains class c,

$$\kappa_c = \frac{\sum_{k \in H_c} q_k f_{kc}}{\sum_{k \notin H_c} q_k}, \tag{1}$$

and the row drifts toward zero in every coordinate when $\kappa_c < 1$. This count of Adam steps is a heuristic, not a bound: it ignores the moments Adam carries between rounds (Appendix A). Centrally the same decay is harmless, because Adam's second-moment estimate for r_c is dominated by data gradients as long as the class appears more often than the estimate forgets.

Figure 1 shows the effect on GeFL-F: after Stage II the conditioning rows of the rarest classes are several times shorter than those of the most frequent classes. Two remedies follow. Decoupling the decay removes the shrinkage. Averaging each row over its holders only, with weights that grow with the holder's count (class-count weighted averaging, CCWA), removes the non-holders' term $\bar{D}_c$ from Proposition 1 and keeps coupled decay:

$$r_c \leftarrow \frac{\sum_k E(n_{kc})\, r_{k,c}}{\sum_k E(n_{kc})}, \qquad E(n) = \frac{1 - \beta^n}{1 - \beta}, \quad \beta = 0.999, \tag{2}$$

where $E(n)$ is the effective number of samples [38]: zero for a client without the class, about n for small n, and bounded by $1/(1-\beta)$ so that one large client cannot dominate a class. Both sums in Eq. (2) are computed by secure aggregation.

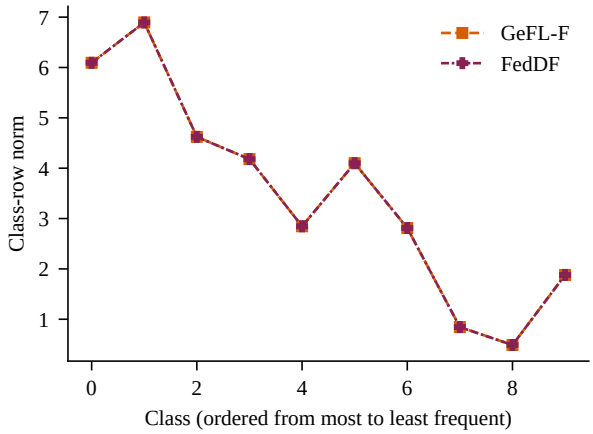


Figure 1: L2 norm of each class's conditioning row of GeFL-F's generator after Stage II, classes ordered from the most to the least frequent (CIFAR-10-LT, imbalance ratio 100, 10 clients, Dirichlet $\alpha = 0.5$; mean over seeds 3–5). FedDF uses the same generator. FedECS-LT's anchored generator has no class rows.

4.2. The last layer: logit corrections are reparametrisations

Proposition 3 (logit corrections are reparametrisations)**.** *Let a head have a free last-layer bias b, trained by SGD without decay on b, and let $a_k \in \mathbb{R}^C$ be a constant of client k, as in logit adjustment ($a_k = \tau \log \pi_k$) [15], FedLC [16] or Balanced Softmax [17]. Training with the loss $\ell(f + a_k, y)$ from (θ_0, b_0) produces exactly the trajectory of training with $\ell(f, y)$ from $(\theta_0, b_0 + a_k)$: every parameter except b is the same at every step. FedRS, which scales the logits of absent classes by $\alpha \in (0, 1]$ [18], is cross-entropy in the rescaled last-layer variables $\alpha w_c, \alpha b_c$ of those classes, trained with the smaller learning rate $\alpha^2\eta$: it changes the initialisation and the step size of the absent classes' rows, not what the client knows about them.*

For a head trained on one client's real data, a logit shift (logit adjustment, FedLC, Balanced Softmax) is therefore equivalent to a different initialisation of the last-layer bias. This does not mean the representation is the one plain training from the same bias would learn: the logits, and hence the gradients reaching the body, differ along the way. What the proposition does show is that the correction adds no information about the classes the client lacks; its effect on the body is measured by removing logit adjustment from FedECS-LT with and without the readout below (Table 14). The readout itself can be computed exactly.

Proposition 4 (partition invariance)**.** *An estimator that is a function of a sum $S = \sum_k \sum_{i \in D_k} T(x_i, y_i)$, with T fixed in advance, takes the same value for every partition of the pooled data, and equals the centralised estimator.*

This property is known for closed-form federated classifiers [22, 33, 24]; we state it because the method relies on it in several places.

The class-balanced linear discriminant used by FedECS-LT (Section 5.4) is such an estimator, and so are Fed3R [22], FedNCM [23], FedCOF [24] and, for a fixed sampling seed, CCVR's virtual features [21] when computed over all clients. The last layer then stops being a source of loss relative to the same readout computed centrally on the same bodies, and the accuracy with this readout compares the bodies. The readout alone raises GeFL-F's balanced accuracy by +16.6 points on CIFAR-10-LT and +7.2 on CIFAR-100-LT (Table 9). A head trained on one client is an extreme case of minority collapse [51]: for the classes the client lacks, the imbalance ratio is infinite.

4.3. The synthetic data: label-free features penalise the logits' spread

Proposition 5 (label-free synthetic data)**.** *If the synthetic label y is uniform on $\{1, \dots, C\}$ and independent of the generated feature, then for every logit vector f, $\mathbb{E}_y\, \ell(f, y) = \mathrm{LSE}(f) - \bar{f} = \log C + \mathrm{KL}(u \,\|\, \mathrm{softmax} f) = \log C + \frac{1}{2C} \sum_c (f_c - \bar{f})^2 + O(\|f - \bar{f}\|^3)$, where $\bar{f}$ is the mean logit and u the uniform distribution.*

When generated features carry little information about their label, the synthetic phase therefore pulls the logits together instead of teaching the classes a client lacks. Alternating it with the real phase decides what survives: in an illustrative model with quadratic losses (the heads are nonlinear and trained with cross-entropy) the round map converges to a point where the synthetic data alone decide the directions that the client's real loss leaves free (Appendix A). For a head trained on one client's real data, the directions of the absent classes are nearly free in this sense, but not exactly: cross-entropy still pushes their logits down. Logit adjustment keeps the real phase from pushing the absent classes' logits down without bound [52]. Figure 2 shows the first effect on the real pipeline: in every round on CIFAR-10-LT the synthetic phase reduces the spread of the heads' logits on the clients' real features, and it reduces it less for FedECS-LT than for GeFL-F.

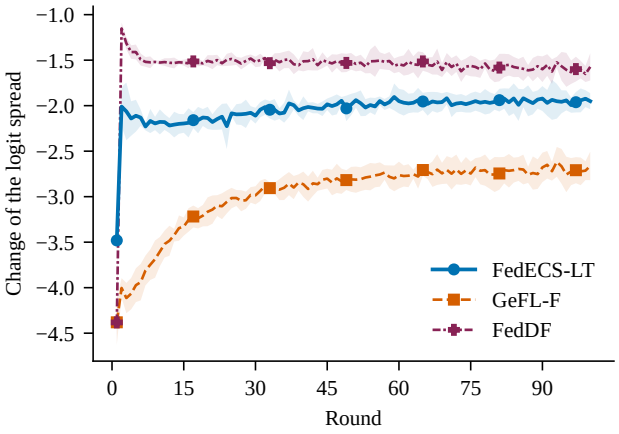


Figure 2: Change of the logit spread (the mean over images of the standard deviation of the logits over classes) of the clients' heads on their own real features during the synthetic phase of each head-training round (after minus before; negative: the synthetic phase pulled the logits together), per round (CIFAR-10-LT, imbalance ratio 100, 10 clients; mean over clients and seeds 3–5, band one standard deviation).

5. FedECS-LT

FedECS-LT keeps the three stages of GeFL-F and the frozen extractor, and changes how every class-specific quantity is obtained. Wherever the data decide a class's result, it uses a statistic that secure aggregation computes exactly instead of a parameter that FedAvg estimates. Algorithm 1 lists the pipeline and Fig. 3 shows it.

5.1. Statistics release

After Stage I, client k computes, for every class c, its count n_{kc}, the sum of its features $S_{kc}^{(1)} = \sum_{i \in D_{kc}} h_i$ and the sum of their squared norms $S_{kc}^{(2)} = \sum_{i \in D_{kc}} \|h_i\|^2$. Secure aggregation returns the totals n_c, $S_c^{(1)}$ and $S_c^{(2)}$, from which the server obtains the exact class means and spreads

$$\mu_c = S_c^{(1)}/n_c, \qquad V_c = S_c^{(2)}/n_c - \|\mu_c\|^2 = \mathbb{E}\big[\|h - \mu_c\|^2 \,\big|\, y = c\big]. \tag{3}$$

These equal the values on the pooled data (Proposition 4) up to the quantisation of secure aggregation: each client's contribution is divided by a public bound and quantised to 30 bits with unbiased stochastic rounding, an error of at most 1.5×10^{-8} of the bound per client and coordinate. Values beyond the bound would be clipped, and with fewer than three participants the sum

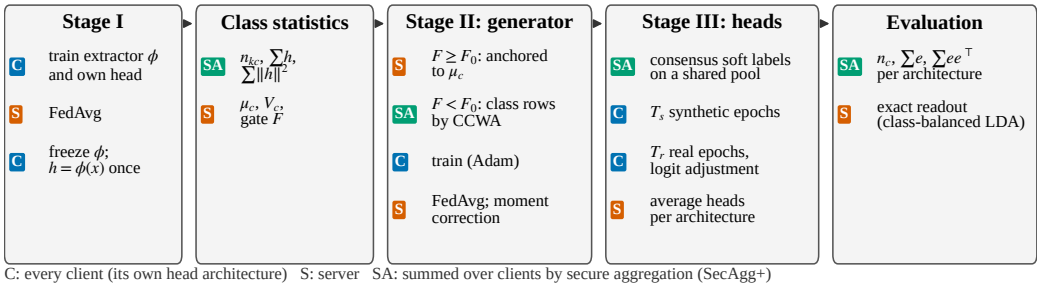


Figure 3: Overview of FedECS-LT, stage by stage. C: computed by every client with its own head architecture; S: computed by the server; SA: summed over the clients by secure aggregation. After Stage I the clients release exact per-class statistics, from which the server computes the class means and spreads and the gate F. The gate chooses the generator: anchored to the class means, or class rows averaged by CCWA. The moment correction is computed once after Stage II. In every round of Stage III the heads train on corrected synthetic features with consensus soft labels and on real features with logit adjustment. For evaluation, every architecture's last layer can be replaced by the exact readout computed from aggregated embedding sums.

is computed in clear, since secure aggregation needs at least three. We call the statistics exact in this sense. Each client uploads them once, $C(d+2)$ numbers for features of dimension d.

5.2. Generator

Separation gate.. A generator anchored to the class means separates two classes by at most a constant times the distance between their means (Proposition 7 in Appendix A). When the means are close relative to the within-class spread, the anchor alone carries little class information. The server therefore computes the class separation

$$F = \frac{\frac{1}{C}\sum_c \|\mu_c - \bar{\mu}\|^2}{\frac{1}{C}\sum_c V_c}, \qquad \bar{\mu} = \frac{1}{C}\sum_c \mu_c, \tag{4}$$

from the released statistics. If $F \geq F_0 = 0.05$ (a threshold fixed during the method's development, before the runs reported here, and not swept) it uses the mean-anchored generator; otherwise it uses GeFL-F's generator with class rows averaged by CCWA (Eq. (2)). On our datasets F ranges from 0.014 to 0.449; the gate chose class rows on CIFAR-10-LT with one shared model and FER-2013.

Mean anchor.. The anchored generator models only the within-class residual around the exact mean,

$$\tilde{h} = \mathrm{ReLU}\big(\mu_y + \mathrm{dec}(z,\, A\mu_y)\big), \qquad z \sim \mathcal{N}(0, I), \tag{5}$$

where the class enters only through its mean and a projection A shared by all classes. The generator has no class-specific parameter, so Proposition 1 has nothing to shrink. It is trained by FedAvg with Adam, as GeFL-F's generator.

Moment correction.. After Stage II the server draws 500 samples per class from the generator, measures their mean $\tilde{\mu}_c$ and spread $\tilde{V}_c$, and corrects every later sample of class c to the exact moments,

$$\tilde{h} \leftarrow \mathrm{ReLU}\big(\mu_c + s_c(\tilde{h} - \tilde{\mu}_c)\big), \qquad s_c = \mathrm{clip}\Big(\sqrt{V_c/\tilde{V}_c},\, 0.1,\, 10\Big). \tag{6}$$

Without the final ReLU and the clipping this is the map of least displacement that attains the exact mean and spread (Proposition 8 in Appendix A); with them, the corrected samples match these moments only approximately: after the correction the generated within-class spread is 0.73 to 0.98 times the exact one (median 0.86) over every class and run of the long-tailed benchmarks. The server sends $(\mu_c, \tilde{\mu}_c, s_c)$ to the clients, which hold the generator and draw corrected samples themselves.

5.3. Heads

Each round, client k first takes $T_s\lfloor n_k/B \rfloor$ SGD steps on batches of B corrected synthetic features ($T_s = 10$ synthetic epochs of the size of its real data; GeFL-F uses $T_s = 1$) and then trains $T_r = 5$ epochs on its real features, each phase with a fresh SGD optimiser. The real phase uses logit adjustment with the client's own prior [15], $\ell(f + \tau \log \hat{\pi}_k, y)$ with $\tau = 1$ and the add-one smoothed class frequencies $\hat{\pi}_k$ of the client. The synthetic phase, whose labels are drawn uniformly over the classes, uses cross-entropy without adjustment, and its target is a *consensus soft label*

$$t(\tilde{h}) = (1 - \omega)\, e_y + \omega\, \frac{1}{G} \sum_{g=1}^{G} \mathrm{softmax}\big(f_g(\tilde{h})\big), \qquad \omega = 0.5, \tag{7}$$

where e_y is the one-hot generated label and f_g the current head of architecture g: the federation's averaged prediction on that feature, the target

form of AvgKD [48] and FedDF's ensemble distillation [30], here computed on generated features through one secure sum [53]. Every client regenerates the same pool of synthetic features from a seed sent by the server and returns $w_k \operatorname{softmax}(f_{a(k)}(\tilde{h}))$ with $w_k = 1/(G\,|\{j : a(j) = a(k)\}|)$; the secure sum over clients is the average in Eq. (7). Since the server holds every head and sends every architecture's current head to every client, this hides nothing beyond what FedAvg already reveals; it only makes the consensus one secure sum. Heads of the same architecture are averaged as in GeFL-F.

5.4. Exact readout

After Stage III, the server broadcasts every architecture's body; each client computes, for every architecture g, the per-class counts, the sums $\sum e$ and the second moment $\sum ee^\top$ of the embeddings e of its real features, and secure aggregation sums them over all clients. With the class means ν_c and the pooled within-class covariance Σ shrunk to $\Sigma_\gamma = (1-\gamma)\Sigma + \gamma\frac{\operatorname{tr}\Sigma}{d_e} I$, $\gamma = 0.1$, the last layer of architecture g becomes the class-balanced linear discriminant

$$w_c = \Sigma_\gamma^{-1}\nu_c, \qquad b_c = -\tfrac{1}{2}\,\nu_c^\top \Sigma_\gamma^{-1}\nu_c, \tag{8}$$

with $b_c = -10^4$ for a class no client holds. By Proposition 4 it equals the same estimator computed centrally on the pooled embeddings of the same body (not a last layer trained centrally by SGD). With the population means and covariance and $\gamma = 0$ it would be the Bayes classifier for equally likely classes when the embeddings of each class are Gaussian with a shared covariance. Computed from the sums, with the shrinkage and a pseudo-inverse, it is a plug-in estimate of that classifier, and otherwise a statistics-based linear classifier without an optimality guarantee. It needs one extra round and $C(d_e+1)+d_e^2$ numbers per architecture and client, and every client must run a forward pass (no training) of every architecture's body on its own features once; a client that cannot do so would be left out of the sums. We report every method of the GeFL-F pipeline both as trained and with this readout (exact readout, ER). The readout is an evaluation device applied to every such method: the headline numbers of FedECS-LT are its heads as trained.

5.5. Cost and privacy

Beyond GeFL-F, FedECS-LT sends the class statistics once, the moment correction once, one vector of C consensus probabilities per synthetic feature per round, and the readout sums once. Table 16 reports the measured

Algorithm 1 FedECS-LT (one run; all sums are computed by secure aggregation)

Stage I: train extractor ϕ and heads by FedAvg; freeze ϕ; $h = \phi(x)$ on every client
clients release $n_{kc}, S_{kc}^{(1)}, S_{kc}^{(2)}$; server forms μ_c, V_c (Eq. (3)) and F (Eq. (4))
Stage II: if $F \geq F_0$ train the anchored generator (Eq. (5)) by FedAvg **else** train GeFL-F's generator, class rows by CCWA (Eq. (2))
server measures $\tilde{\mu}_c, \tilde{V}_c$ and sends the moment correction $(\mu_c, \tilde{\mu}_c, s_c)$ (Eq. (6))
for round $t = 1, \ldots, T_{\mathrm{TN}}$ **do** ▷ Stage III
    server sends heads and a pool seed; clients return the secure sum of consensus predictions (Eq. (7))
    each client: T_s epochs on corrected synthetic features with soft targets $t(\tilde{h})$, then T_r epochs on real features with logit adjustment
    server averages heads of the same architecture
end for
Evaluation: clients release embedding sums per architecture; server sets every last layer by Eq. (8) (reported separately as ER)

communication and time of every method. The server, and through the generator and the moment correction every client, learns the federation-level class means and spreads. For a class that one client holds, these equal that client's own statistics, and they still describe rare classes held by few samples: a class mean over a handful of images reveals more about them than a mean over thousands. Formal differential privacy for the released statistics is possible with the Gaussian mechanism but is not part of the results reported here, and it would not cover the FedAvg training of the generator and heads.

6. Study design

The study tests the analysis of Section 4 and the method of Section 5 on the problem of Section 3: clients with different head architectures and long-tailed, skewed label distributions. This section states the research questions, the system under test, the data, the methods compared, the metrics, the protocol, the ablations, the statistical analysis and the threats to validity.

6.1. Research questions

Table 2 lists the questions and the experiment that answers each.

Table 2: Research questions and the experiments that answer them. All experiments use the same client splits for every method.

Question	Experiment	Reported in
RQ1 diagnosis	class-row norms, readout gain and logit spread in GeFL-F's runs	Section 4
RQ2 effect	main comparison, 10 clients with ten architectures; 50 clients	Section 7.1
RQ3 representation	every GeFL-F-pipeline method with the exact readout	Section 7.1
RQ4 components	variants of GeFL-F; FedECS-LT with one component changed	Section 7.5
RQ5 scope	natural imbalance; sweeps of K, α, ρ; one shared model	Sections 7.3 and 7.4, Table 15

- **RQ1 (diagnosis).** Where does GeFL-F lose the rare classes under long-tailed label skew? The analysis names three places: the generator's class rows, the heads' last layers and the synthetic phase. RQ1 checks each in GeFL-F's own runs.
- **RQ2 (effect).** Does FedECS-LT raise balanced accuracy over GeFL-F and the other methods that work with different client models, with every model evaluated as trained?
- **RQ3 (representation or last layer).** How much of the difference remains when every method of the GeFL-F pipeline gets the same exact readout, so that only the heads' bodies differ?
- **RQ4 (components).** Which changes to GeFL-F carry the effect?
- **RQ5 (scope).** Does the effect hold with natural class imbalance, with other client numbers, label skews and imbalance ratios, and in the standard protocol with one model shared by all clients?

Cost and calibration are reported for every method as secondary outcomes.

6.2. System under evaluation

The system is GeFL-F's three-stage pipeline (Section 3). In Stage I the clients train a shared feature extractor and their heads by federated averaging; the extractor is then frozen, and every image's features are computed once without augmentation. In Stage II the clients train a conditional feature generator by federated averaging. In Stage III every client trains its head, first on generated features and then on its real features, and the heads of each architecture are averaged.

FedECS-LT keeps this pipeline and changes five things (Section 5):

1. the generator, which is anchored to the exact class means, or uses class rows averaged by CCWA when the gate finds the classes poorly separated;
2. the moment correction of generated features;
3. logit adjustment in the real phase;
4. consensus soft labels in the synthetic phase;
5. the number of synthetic epochs per round, $T_s = 10$ instead of GeFL-F's 1.

The exact readout of Section 5.4 is computed after training. We report it separately for every method of the pipeline and never mix it into the numbers as trained. Table 3 lists the settings of FedECS-LT, which are the same on every dataset.

The extractor is one convolutional block followed by 2×2 max pooling: one 3×3 convolution with three output channels on MNIST, Fashion-MNIST, SVHN, OrganAMNIST, EMNIST and FER-2013, and two convolutions with three and ten output channels on CIFAR-10, CIFAR-100, DermaMNIST, BloodMNIST, PathMNIST and GTSRB, each convolution followed by batch normalisation and ReLU. The ten heads, CNN-1 to CNN-10, have one to four convolutional blocks of different widths and a linear last layer (Appendix B). They follow GeFL-F [8]; the widths do not increase monotonically from CNN-1 to CNN-10.

6.3. Datasets and client splits

Long-tailed benchmarks.. CIFAR-10 [54] and CIFAR-100 [54] are made long-tailed by our construction: GeFL-F's fraction of the training set [8] is divided over the classes by the profile of Section 3 with $\rho = 100$ (GeFL's own splits are balanced). The fraction is GeFL-F's on MNIST, Fashion-MNIST and SVHN (0.1) and on CIFAR-10 (0.5). GeFL does not use CIFAR-100; there we take 0.22, which gives the usual CIFAR-100-LT shape. No class receives more images than remain after the validation images are held out. On CIFAR-10-LT and CIFAR-100-LT the profile asks for more than that for the largest classes, which are therefore capped. So the realised ratio between the largest and the smallest class is below 100 (Table 4); we name every setting by its ρ. The class index orders the classes from the most to the least frequent, so the tail classes are the same in every run.

Table 3: Settings of FedECS-LT (all datasets; with one shared model, 10 real epochs per round and SGD with momentum 0.9 and the protocol's learning-rate decay). GeFL-F uses $T_s = 1$, cross-entropy and no consensus labels; the generator settings are GeFL-F's.

Setting	Value
Separation gate threshold F_0	0.05
CCWA parameter β	0.999
Moment correction: samples per class	500
Moment correction: range of s_c	$[0.1, 10]$
Synthetic / real epochs per round T_s / T_r	10 / 5
Logit adjustment τ	1
Consensus soft-label weight ω	0.5
Readout shrinkage γ	0.1
Generator optimiser, learning rate	Adam, 10^{-3}
Generator weight decay (coupled L2)	10^{-3}
Head optimiser (each phase, each round)	SGD, learning rate 0.1, no momentum

Natural imbalance.. DermaMNIST [55], OrganAMNIST [55], BloodMNIST [55], PathMNIST [55], EMNIST (ByClass) [56] and FER-2013 [57] keep their own class sizes, with training sets capped at 20,000 images. Their classes are relabelled by training frequency. On three of them the realised ratio is small (Table 4): their results speak to label skew more than to a long tail.

Splits.. The training images are split among $K = 10$ clients by a Dirichlet draw with $\alpha = 0.5$ per class. The draw is repeated, at most 100 times, until every client holds at least ten images; if it still fails, random images move from the largest to the smallest client. Client k trains head architecture k, so in the main setting each architecture has one client.

A held-out block of training images that no client holds forms a validation set: $\min(200, \lfloor 2000/C \rfloor)$ images per class, and at most a quarter of a class with natural imbalance. It has two uses: it trains the referee classifier of the generator diagnostics, and it sets the public bounds by which secure aggregation scales the released vectors. It is never used to select a model or a hyperparameter. The other unused training images are discarded. Every result is on the official test set. A seed fixes the split, so all methods see the same clients; we checked that the class counts of every client are identical across methods for every dataset and seed.

Table 4: Datasets as used: classes, training images held by the clients, test images, and the realised ratio between the largest and the smallest class in the federation (natural imbalance: the dataset's own class sizes).

Dataset	Classes	Training images	Test images	Largest / smallest class
CIFAR-10-LT	10	18,485	10,000	48
CIFAR-100-LT	100	10,926	10,000	96
DermaMNIST	7	6,231	2,005	75
OrganAMNIST	11	19,994	17,778	5
BloodMNIST	8	10,359	3,421	3
PathMNIST	9	19,997	7,180	2
EMNIST (ByClass)	62	19,970	116,323	20
FER-2013	7	19,997	3,589	17

6.4. Protocols

Main setting: different client models.. This is GeFL-F's setting with $K = 10$ clients, one per architecture. The rounds of the three stages are GeFL-F's: 50, 200 and 100 on CIFAR-10, and the same on CIFAR-100 (Appendix B lists every dataset). All federated methods of this setting share the same Stage-I extractor for a given dataset and seed: Stage I is computed once and reused. GeFL's second setting has $K = 50$ clients, five per architecture (Section 7.1); every method of the main comparison is run there too.

One shared model.. We follow the FedLTLib protocol [28], whose training settings come from CReFF [9]: CIFAR-10-LT and CIFAR-100-LT with the standard profile $n_c = \lfloor n_{\max} \rho^{-c/(C-1)} \rfloor$ at $\rho \in \{100, 50, 10\}$, where $n_{\max}$ is the number of images per class left after 200 per class (20 on CIFAR-100) are held out for validation. Rounding down gives the smallest CIFAR-100 class 4 images at $\rho = 100$.

There are 20 clients, Dirichlet $\alpha = 0.5$, and 8 clients (40%) take part in each round. All share one ResNet-8 [58], trained for 200 rounds of 10 local epochs with batch size 32 and SGD (learning rate 0.1, momentum 0.9, decay 0.99 per round). The baselines run the authors' LongTailLib training code unchanged, on our splits, with the backbone set to ResNet-8 for every method (the library defaults to ResNet-18 or ResNet-20 for FedLoGe, Fed-GraB, FedYoYo and FedETF); we evaluate them every fifth round and score Fed-GraB with its trained classifier.

For FedECS-LT the ResNet-8 is split after its second stage into a frozen extractor and a head. The extractor is trained for 100 rounds with the proto-

col's local updates, the generator for 100 rounds, and the head for 100 rounds with 10 real epochs per round; the learning-rate decay continues from the extractor's rounds. So FedECS-LT uses the baselines' 200 training rounds plus 100 generator rounds. Its head trains on frozen features of images without augmentation, whereas the baselines train end to end with augmentation. With one architecture, the consensus soft label is the current shared model's own prediction. FedECS-LT averages the clients' models with equal weights, as in the main setting; the benchmark's FedAvg weights them by their numbers of samples. The clients of each round are a seeded subset; the library draws its own, so participation is not paired between FedECS-LT and the baselines.

6.5. Methods compared

Table 5 lists every method, its reference and the origin of its code, and Table 6 what each method of the main setting exchanges and computes. The methods fall into six groups, each chosen for one question.

- **GeFL-F**, the method FedECS-LT extends: the difference to it answers RQ2 directly.
- **FedDF** [30], ensemble distillation on the server. GeFL distils on public data; we run FedDF's distillation on GeFL-F's generated features instead, so that no method uses extra data.
- **Per-client long-tail losses**: Balanced Softmax [17], FedLC [16] and FedRS [18]. They test whether a correction of the loss alone repairs the long tail. They are applied to GeFL-F without its generator. At $K = 10$ each architecture has one client, so their heads train on that client's real features only and exchange nothing in Stage III.
- **GeFL's baselines**: FedAvg [1], FedProx [45] and FedALA [47] run within each architecture group, and AvgKD [48] distils across all architectures, as GeFL compares them [8]. CCVR [21] and FedETF [34], two long-tail methods, are applied within each group. At $K = 10$ each group has one client, so FedAvg, FedProx and FedALA exchange nothing in Stage III; AvgKD and CCVR still use the other clients' predictions or statistics.
- **Model-heterogeneous methods from HtFLlib** [27]: prototypes (FedProto, FedTGP), a generator of embeddings (FedGen), a shared

global head (FedGH), shared last layers (LG-FedAvg) and shared sub-networks (FedMRL). They represent the other ways in which clients with different models exchange knowledge.

- **References**: Local, in which each client trains its extractor and head alone, is the reference without federation. The pooled-data reference trains the same architectures with cross-entropy on all training data pooled, without a generator and without a long-tail correction.

Methods designed for one shared model (Table 5, lower part) are compared in the one-shared-model setting only.

Settings and tuning.. No baseline was tuned on our data; their settings come from different sources:

- the HtFLlib methods use the values of HtFLlib's run scripts (SGD, learning rate 0.01, batch 10, feature dimension 512), with 5 local epochs per round to match our head stage;
- the LongTailLib methods use the benchmark's common settings, which differ from some methods' own papers;
- the grouped baselines use the settings of GeFL's code (FedAvg, FedProx with $\mu = 0.01$, AvgKD) and of the authors' code (FedALA) or LongTailLib (CCVR, FedETF);
- Balanced Softmax and FedLC ($\tau = 1$) are re-implemented losses with the client's add-one smoothed class counts, and FedRS ($\alpha = 0.5$) with the classes the client lacks;
- FedDF uses 20 server distillation steps per round (Appendix B).

The settings of FedECS-LT (Table 3) were fixed during its development, on earlier runs of the same benchmarks with seeds 0 to 5. These included seeds 3 to 5 with the same client splits as the reported runs, and every earlier run was evaluated on the same test sets. The settings were not changed for the runs reported here. This can favour FedECS-LT over methods that were not tuned (Section 6.10).

Table 5: Methods compared, their protocol and the origin of their code. HtFLlib [27] (Apache-2.0) methods are ported to Flower with HtFLlib's settings and 5 local epochs per round, on the shared frozen extractor. LongTailLib [28] methods run in the authors' code, downloaded at a fixed commit. FedAvg, FedProx and AvgKD of the main setting follow GeFL's code [8] (MIT).

Method	Venue	Protocol	Code
GeFL-F [8]	IEEE TMC 2025	different models	re-implementation of the authors' algorithm
FedDF [30]	NeurIPS 2020	different models	re-implementation: server distillation on generated features
Balanced Softmax [17]	NeurIPS 2020	different models	loss on each head, no generator
FedLC [16]	ICML 2022	both	loss on each head, no generator / LongTailLib
FedRS [18]	KDD 2021	different models	loss on each head, no generator
FedAvg, FedProx (grouped) [1, 45]		different models	GeFL's code, within each architecture
AvgKD [48]	ICLR 2022	different models	GeFL's code, across architectures
FedALA (grouped) [47]	AAAI 2023	different models	authors' ALA module, within each architecture
CCVR, FedETF (grouped) [21, 34]		different models	LongTailLib's version, within each architecture
FedProto [3]	AAAI 2022	different models	HtFLlib
FedTGP [4]	AAAI 2024	different models	HtFLlib
FedGen [5]	ICML 2021	different models	HtFLlib
FedGH [6]	ACM MM 2023	different models	HtFLlib
LG-FedAvg [31]	arXiv 2020	different models	HtFLlib
FedMRL [7]	NeurIPS 2024	different models	HtFLlib
Local		different models	each client alone
FedAvg [1], FedProx [45], MOON [46]		one shared model	LongTailLib
CCVR [21], CReFF [9], FedETF [34]		one shared model	LongTailLib
Fed-GraB [10], FedLoGe [11]		one shared model	LongTailLib
FedYoYo [12], RUCR [13]		one shared model	LongTailLib
FedECS-LT	this work	both	ours (Flower)

6.6. Metrics

Let r_{gc} be the recall of class c by the model of architecture g on the official test set. The test set is balanced on Fashion-MNIST, CIFAR-10 and CIFAR-100 and imbalanced on MNIST, SVHN and the naturally imbalanced datasets; balanced accuracy weights every class equally in either case. The primary metric is the balanced accuracy, $\frac{1}{G}\sum_g \frac{1}{C}\sum_c r_{gc}$: the mean over the $G = 10$ client models of each model's mean class recall. GeFL [8] also averages over the ten models, with top-1 accuracy at each model's best round. It is reported with the heads as trained and, for the methods of the GeFL-F pipeline, with the exact readout. The second number answers RQ3.

The other metrics are:

- the recall of the tail classes, the last 40% by frequency rank, and of the worst class, $\min_c \frac{1}{G}\sum_g r_{gc}$, which is the lowest class recall of the recall averaged over the models;
- the number of classes that no model recognises (pooled recall zero);
- macro-F1, macro precision and the multi-class Matthews correlation coefficient [59];

Table 6: What each method of the main setting uses (CIFAR rounds; per-round epochs of Stage III). Stage I is shared by all federated methods. Heads of the same architecture are averaged, which exchanges something only when an architecture has several clients (not at $K = 10$); the column gives what is exchanged beyond that.

Method	Generator (rounds)	Synthetic / real epochs	Exchange in Stage III	Exact readout	Loss (real phase)
FedECS-LT	anchored or CCWA (200)	10 / 5	consensus labels	yes	logit-adjusted
GeFL-F	class rows, flat (200)	1 / 5	none at $K = 10$	yes	cross-entropy
FedDF	GeFL-F's (200)	1 / 5	server distillation	yes	cross-entropy
Balanced Softmax, FedLC, FedRS	none	0 / 5	none at $K = 10$	yes	per-client correction
HtFLlib methods	FedGen: in every batch	0 / 5	prototypes, heads or sub-networks	no	method's own
Grouped FedAvg, FedProx, FedALA	none	0 / 5	heads (none at $K = 10$)	no	method's own
AvgKD; grouped CCVR, FedETF	none	0 / 5	predictions or statistics	no	method's own
Local	none	alone: $5T_{\mathrm{FE}} + 5T_{\mathrm{TN}}$ epochs	none	no	cross-entropy

- the geometric mean of the class recalls, each floored at 1%, computed for each model and then averaged;
- macro AUC-PR;
- the expected calibration error with 15 bins [60] and its class-wise form [61];
- the balanced accuracy of the weakest architecture;
- the accuracy on each client's own class mix: the test accuracy reweighted to the class frequencies of that model's clients.

Every per-class metric is logged for every class at every evaluation.

GeFL's code reports each model's best test accuracy over the evaluated rounds. We report the model after the last round, which involves no selection on the test set. GeFL's measure is given in Table C.19.

Cost is measured as follows:

- communication: the bytes of the model and statistics messages per client, with a secure sum counted as eight bytes per summed value and client, without the protocol's key and share messages and without the test evaluation; the readout's sums are included;
- time: the wall-clock time of a run on a T4 GPU, which in most notebooks ran a second run at the same time;
- computation: forward floating-point operations per image, with training counted as three forward passes.

6.7. Protocol and implementation

Every configuration is run with seeds 3, 4 and 5; settings that came back with fewer seeds say so. A seed fixes the split and the initialisation of every method, so runs with the same seed are paired. Models are evaluated every five rounds on the test set, before and after aggregation. The reported model is the one after the last round.

FedECS-LT and every method of the main setting run in Flower 1.42 [26], a simulation with the Ray backend, one simulation per run. The one-shared-model baselines run in the authors' LongTailLib code on the same client splits and are evaluated on the same test set. All runs used Kaggle notebooks with two NVIDIA T4 GPUs, in most notebooks with two runs per GPU at a time.

All releases of FedECS-LT use Flower's SecAgg+ [20], with 30-bit stochastic quantisation of every vector after division by a public bound: a multiple of the federation size and of the largest value on the validation features for the class and readout sums, and fixed bounds for the counts, the CCWA rows and the consensus predictions. Values beyond eight times the bound would be clipped.

The Flower implementation was checked against our earlier, single-process implementation, which follows GeFL's code. The client splits are identical, every client training step agrees weight for weight in unit tests, and the balanced accuracy agrees within the larger of two points and three seed standard deviations (Appendix B).

GPU computation is not bit-reproducible (cuDNN autotuning and non-deterministic GPU kernels), so two runs with the same seed differ. A notebook run twice with the same run list (BloodMNIST, 39 runs) measures this. The standard deviation of the difference between the two runs of one configuration is 1.2 points of balanced accuracy as trained and 0.6 with the exact readout.

6.8. Ablations

Two sets of runs separate the components (RQ4), on CIFAR-10-LT and CIFAR-100-LT with the same splits and seeds as the main comparison.

- **Variants of GeFL-F** test the diagnosis: GeFL-F without its generator (what the generator adds at all), GeFL-F with decoupled decay and logit adjustment (the cheapest repair of the first two losses), GeFL-F with a Gaussian class-conditional generator from the exact statistics (a CCVR-style sampler [21]), and FedDF with logit adjustment.

- **FedECS-LT with one component changed** measures each component's share:
  - class rows with CCWA, or with flat averaging, instead of the gate;
  - no moment correction;
  - no consensus soft labels;
  - no logit adjustment;
  - one synthetic epoch per round instead of ten. This also separates the effect of the longer synthetic phase from that of the exact statistics.

Each variant is reported as trained and with the exact readout.

6.9. Statistical analysis

We report the mean ± sample standard deviation over seeds. A difference to FedECS-LT is the mean paired difference over the seeds both methods have, with a 95% t interval and the paired t-test. The p-values are corrected by Holm's procedure [62] over the other methods of the same table and setting: every method as trained, and the methods of the GeFL-F pipeline with the exact readout. Local is included; the pooled-data reference and the diagnostic runs are not. There is no correction across datasets or metrics. "Best" and "strongest other method" are decided on the seeds both methods have.

With three seeds the tests have little power, and they rest on the t model: a sign-flip permutation test cannot give $p < 0.25$ with three paired differences. A verdict near $p = 0.05$ can also depend on which methods form the Holm family. GPU non-determinism alone gives the mean of three paired differences a standard deviation of about half the two-run value above; we treat differences smaller than twice that, 1.4 points as trained and 0.7 with the exact readout, as unresolved, and report every interval with its test.

6.10. Threats to validity

Internal validity..

- **Tuning.** The settings of FedECS-LT were chosen on earlier runs of the same benchmarks, partly with the same seeds and client splits and always on the same test sets, while the baselines were not tuned. The reported numbers may therefore favour FedECS-LT; there is no held-out test set that the development never saw.

- **Compute.** FedECS-LT trains its heads for ten synthetic epochs per round, against one for GeFL-F and FedDF and none for the methods without a generator (Table 6), and it communicates more (Table 16). The ablation with one synthetic epoch shows how much of the result of FedECS-LT depends on the longer phase; it does not show what GeFL-F or FedDF would reach with ten.

- **Missing runs.** With one shared model, runs of FedECS-LT failed on some seeds and imbalance ratios where the baselines finished, so the missing runs are not random; Table 12 gives the seeds that came back.

- **Timing.** Wall-clock times were measured with a second run on the same GPU in most notebooks and are only indicative.

- **Ports.** The HtFLlib methods were designed for end-to-end training and run here on a frozen extractor with HtFLlib's settings. Section 7.1 shows that FedProto and FedTGP lose accuracy after their first evaluated rounds, which may mean that these settings do not suit frozen features.

- **Non-determinism.** Pairing by seed removes the variation of the split but not that of GPU computation.

Construct validity..

- Balanced accuracy is a mean over ten models of different sizes, so a large gain of the small heads and a small gain of the large heads give the same number. Table C.22 gives the weakest architecture.

- The headline numbers are the heads as trained; the readout is a separate evaluation.

- The server holds a small validation set (held out from the training data) for the generator diagnostics and the secure-aggregation bounds; a deployment without such data would need another source for the bounds.

- The last-round model can be below a method's best round; Table C.19 gives the best round.

External validity..

- The extractor is one frozen convolutional block and the heads are small CNNs, as in GeFL-F. Absolute accuracies are therefore low, and the results may not carry over to large pretrained models.
- The long tail is synthetic on the benchmarks, with the same tail classes in every run.
- The label skew is a Dirichlet split; feature shift between clients is not studied.
- The one-shared-model protocol is run at fewer seeds for FedECS-LT than planned (Table 12).

7. Results

7.1. Main comparison: different client models

Table 7 gives the balanced accuracy of every method on the long-tailed benchmarks (CIFAR-10 and CIFAR-100) with the heads as trained, and Table 8 the paired difference of FedECS-LT to each method. FedECS-LT has the highest balanced accuracy on both benchmarks. Its margin over the strongest other method is +5.2 points on CIFAR-10-LT (over FedDF) and +4.3 on CIFAR-100-LT (over FedDF). With three seeds the difference is significant after Holm's correction on CIFAR-100-LT ($p = 0.021$ against FedDF) but not on CIFAR-10-LT ($p = 0.066$), although every seed favours FedECS-LT. On CIFAR-100-LT, 0.3 classes on average are recognised by none of the ten client models of FedECS-LT, against 12.3 for FedDF. Against GeFL-F, the method it extends, the difference is +9.1 points [+4.8, +13.3] on CIFAR-10-LT and +6.6 [+5.6, +7.6] on CIFAR-100-LT. The model-heterogeneous methods that exchange prototypes or shared sub-networks (FedProto, FedTGP, FedGH, LG-FedAvg, FedMRL) were designed for label skew without a global long tail; under the long tail their balanced accuracy is close to or below that of Local, because nothing in them gives an architecture information about the classes its clients lack. FedProto, FedTGP and FedGen are best in the first rounds and lose 5.7 ± 1.0, 6.7 ± 0.7 and 5.3 ± 0.9 points of accuracy by the last round on CIFAR-10-LT (2.8 ± 0.5, 3.2 ± 0.3 and 3.1 ± 1.2 on CIFAR-100-LT); they are also the weakest on their clients' own class mixes, on which HtFLlib scores such personal models (Table C.20). FedECS-LT, too, is past

Table 7: Balanced accuracy (%) on the test set, heads as trained; mean ± standard deviation over seeds 3–5. Long-tailed benchmarks (imbalance ratio 100), 10 clients with ten different head architectures, Dirichlet label split ($\alpha = 0.5$). Best in each column in bold.

Method	CIFAR-10	CIFAR-100
FedECS-LT	**42.8 ± 2.0**	**14.3 ± 0.6**
GeFL-F	33.7 ± 1.2	7.7 ± 0.4
Balanced Softmax	36.3 ± 1.2	9.0 ± 0.3
FedLC	34.3 ± 1.1	8.5 ± 0.3
FedRS	34.1 ± 1.1	8.5 ± 0.3
FedDF	37.5 ± 1.2	10.0 ± 0.5
Local	32.1 ± 0.7	7.9 ± 0.8
FedProto	27.1 ± 0.8	4.8 ± 0.3
FedTGP	25.3 ± 1.2	4.5 ± 0.6
FedGen	26.9 ± 2.0	4.5 ± 1.3
FedGH	31.9 ± 1.1	6.5 ± 0.1
LG-FedAvg	31.3 ± 0.8	6.3 ± 0.1
FedMRL	33.7 ± 1.0	8.3 ± 0.2

its best round at the last one, by 3.4 ± 1.2 points on CIFAR-10-LT and 0.9 ± 0.3 on CIFAR-100-LT; Table C.19 gives every method's best round, as GeFL reports it. Balanced accuracy has a price on the clients' own class mixes. There FedECS-LT reaches 68.6 ± 1.9 on CIFAR-10-LT and 31.5 ± 0.3 on CIFAR-100-LT, below 8 of the 11 other federated methods on CIFAR-10-LT and 5 of 11 on CIFAR-100-LT; Balanced Softmax, FedLC, FedRS, FedDF, Local and FedMRL are above it on both (Table C.20).

Figure 4 shows the paired differences on CIFAR-10-LT with their confidence intervals.

With the exact readout.. Most of GeFL-F's long-tail loss is in its last layer (Section 4.2). Table 9 therefore compares the methods of the GeFL-F family again with every last layer replaced by the same exact readout, so that the remaining differences are those of the bodies. With the same readout the gap to every method shrinks: FedECS-LT remains +2.3 points ahead of GeFL-F on CIFAR-10-LT and +3.8 on CIFAR-100-LT, and +1.4 and +1.5 points ahead of the strongest other method (Balanced Softmax and FedRS). The differences to the strongest other method are not significant after Holm's correction ($p = 0.37$ and 0.058), although on CIFAR-100-LT the uncorrected

Table 8: Balanced accuracy of FedECS-LT minus each method (points), paired over seeds 3–5, with the 95% t confidence interval; bold where the paired t-test is significant after Holm's correction over the methods ($p < 0.05$). Same setting as Table 7.

Method	CIFAR-10	CIFAR-100
GeFL-F	+9.1 [+4.8, +13.3]	**+6.6 [+5.6, +7.6]**
Balanced Softmax	+6.4 [+2.1, +10.8]	**+5.3 [+4.2, +6.5]**
FedLC	+8.5 [+4.7, +12.3]	**+5.8 [+4.6, +7.0]**
FedRS	+8.7 [+4.9, +12.4]	**+5.8 [+4.7, +7.0]**
FedDF	+5.2 [+1.2, +9.3]	**+4.3 [+2.7, +5.9]**
Local	+10.7 [+5.5, +15.9]	**+6.4 [+4.1, +8.6]**
FedProto	**+15.7 [+12.6, +18.7]**	**+9.6 [+8.5, +10.6]**
FedTGP	**+17.5 [+14.1, +20.9]**	**+9.8 [+8.8, +10.8]**
FedGen	**+15.9 [+13.3, +18.5]**	**+9.8 [+5.8, +13.7]**
FedGH	+10.9 [+6.6, +15.2]	**+7.8 [+6.7, +8.9]**
LG-FedAvg	+11.4 [+7.6, +15.3]	**+8.0 [+7.0, +9.1]**
FedMRL	+9.1 [+5.3, +12.8]	**+6.0 [+5.0, +7.0]**

95% interval [+0.6, +2.4] excludes zero (CIFAR-10-LT: [-0.7, +3.6]). On CIFAR-100-LT the differences to GeFL-F and FedDF remain significant with the readout ($p = 0.0095$ and 0.0143). Most of the margin as trained is therefore in the last layer, and three seeds do not show whether FedECS-LT also learns a better representation than the strongest other method.

7.2. Rare classes

Table 10 reports the recall of the tail classes, and Fig. 5 the recall of every class against its number of training images. Against GeFL-F, the gain of FedECS-LT on CIFAR-10-LT is in the tail: its recall rises by +17.5 points on the tail classes and by +3.0 on the head classes. On CIFAR-100-LT the gain is spread over all three groups (head +7.0, medium +8.7, tail +4.7 points); its tail recall is 6.4 ± 0.3 against 1.7 ± 0.1, and its worst-class recall 0.2 ± 0.2 against 0.0 ± 0.0. Against FedDF, FedECS-LT gains on the tail classes (+20.8 points on CIFAR-10-LT) and loses on the head classes (-8.1). With the exact readout the tail gain over Balanced Softmax is +0.0 on CIFAR-10-LT and -0.1 on CIFAR-100-LT: the tail gain over the loss-only methods comes from the last layer. Figure 6 splits the recall into the head, medium and tail classes.

Table 9: Balanced accuracy (%) with every architecture's last layer replaced by the exact class-balanced readout (Section 5.4); mean ± standard deviation over seeds 3–5. Methods of the GeFL-F pipeline (same extractor, heads with a CNN body and a linear last layer); the HtFLlib methods and Local were not evaluated with the readout. Same setting as Table 7.

Method	CIFAR-10	CIFAR-100
FedECS-LT	**52.6 ± 1.3**	**18.7 ± 0.2**
GeFL-F	50.3 ± 1.2	14.9 ± 0.2
Balanced Softmax	51.2 ± 0.7	17.2 ± 0.3
FedLC	51.2 ± 0.7	17.2 ± 0.3
FedRS	51.1 ± 0.7	17.2 ± 0.3
FedDF	51.0 ± 1.2	15.8 ± 0.3

7.3. Datasets with natural class imbalance

Table 11 repeats the comparison on six datasets whose class sizes are their own (DermaMNIST, OrganAMNIST, BloodMNIST, PathMNIST, EMNIST (ByClass) and FER-2013). FedECS-LT has the highest mean balanced accuracy as trained on all six, with margins of +3.4 (FER-2013) to +12.3 points (EMNIST (ByClass), one seed) over the strongest other method; the margin is significant after Holm's correction on two of the five datasets with three seeds. With the exact readout for every method of the GeFL-F pipeline the margins are below one point, close to the noise floor of 0.7 points of Section 6.9, and on BloodMNIST FedDF is level with FedECS-LT. The separation gate chose the class-row generator on FER-2013, where the class means are too close for an anchor.

7.4. One shared model

Table 12 gives the results in the FedLTLib protocol. Here several long-tail methods are stronger: FedYoYo, FedLoGe, CCVR and FedETF are above FedECS-LT on CIFAR-10-LT and FedYoYo, FedLoGe and CCVR on CIFAR-100-LT, and 7 and 8 methods are below it. At imbalance ratio 100, FedECS-LT reaches 61.3 ± 1.8 on CIFAR-10-LT and 34.1 on CIFAR-100-LT, about the level of FedAvg, while the strongest method, FedYoYo, is 16.8 and 8.9 points ahead of FedECS-LT on the seeds both have (3 and 4 on CIFAR-10-LT, 4 on CIFAR-100-LT; Table 12 averages every seed of each method). With one shared model nothing prevents training the whole network end to end on real, augmented data, whereas the head of FedECS-LT trains on frozen

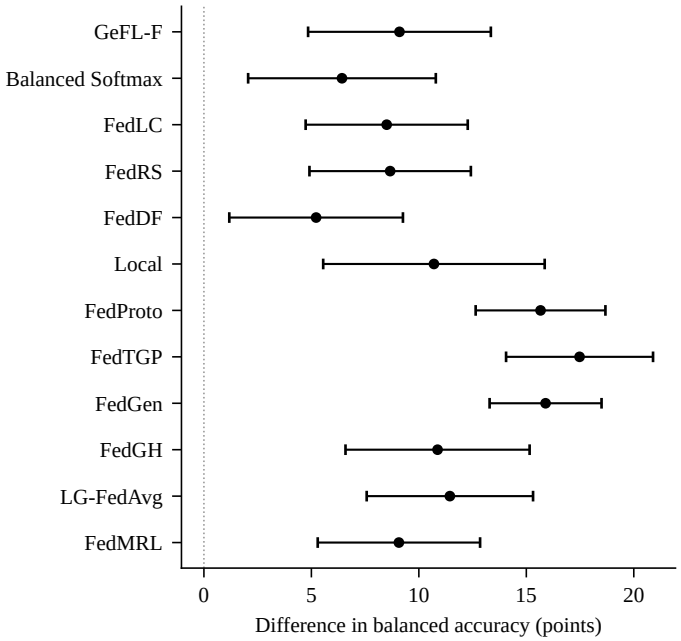


Figure 4: Balanced accuracy of FedECS-LT minus that of each method on CIFAR-10-LT (points; mean of the paired differences over seeds 3–5 with 95% t confidence interval; right of the dotted line FedECS-LT is better). Imbalance ratio 100, 10 clients, Dirichlet $\alpha = 0.5$.

features of images without augmentation, from an extractor trained for half of the rounds. We did not measure the representations in this protocol, so this explanation is not tested. The generator, which is what lets clients with different models exchange knowledge, is not needed when the models can simply be averaged. Here the exact readout changes the balanced accuracy of FedECS-LT by -3.0 points on CIFAR-10-LT and -0.1 on CIFAR-100-LT. Training the extractor of FedECS-LT for all 200 rounds changes its balanced accuracy on CIFAR-10-LT by **[t2/cifar10-r100/fe200_diff]** points.

Comparison with published numbers.. Our baselines run in the authors' code on our splits and report the model after the last round. Against FedYoYo's evaluation in the same protocol [12], which trains 300 rounds, most methods are within about three points of the published values (e.g. FedAvg 58.2, CCVR 68.1, FedETF 68.3, FedYoYo 81.5 on CIFAR-10-LT). FedLTLib [28] reports the best round and gives higher values than ours for every method. Three methods are clearly below the published values on CIFAR-10-LT in our runs: CReFF (70.5 in [12]), Fed-GraB (70.6) and FedLC (69.0). They run in

Table 10: Recall of the tail classes (the 40% least frequent; %), heads as trained; mean ± standard deviation over seeds 3–5. Same setting as Table 7.

Method	CIFAR-10	CIFAR-100
FedECS-LT	**31.4 ± 2.7**	**6.4 ± 0.3**
GeFL-F	13.9 ± 1.2	1.7 ± 0.1
Balanced Softmax	20.0 ± 1.9	2.2 ± 0.1
FedLC	16.1 ± 1.4	1.7 ± 0.1
FedRS	15.7 ± 1.4	1.7 ± 0.1
FedDF	10.6 ± 1.0	0.8 ± 0.1
Local	14.0 ± 1.2	1.6 ± 0.2
FedProto	10.7 ± 1.9	0.9 ± 0.0
FedTGP	10.8 ± 1.7	1.2 ± 0.5
FedGen	8.6 ± 1.1	0.7 ± 0.3
FedGH	13.3 ± 0.8	1.1 ± 0.1
LG-FedAvg	13.3 ± 0.6	1.0 ± 0.0
FedMRL	14.9 ± 1.2	1.7 ± 0.1

the benchmark's code with its common settings (learning rate 0.1, batch size 32, ten local epochs), which differ from those in the methods' own papers; we did not tune them and report them as obtained. FedYoYo's evaluation trains 300 rounds and ours 200. Figure 7 shows the balanced accuracy over the rounds.

7.5. Ablations

Variants of GeFL-F.. Table 13 isolates what each change to GeFL-F contributes, on CIFAR-10-LT and CIFAR-100-LT with the same splits and seeds: GeFL-F without its generator (heads on real features only), with logit adjustment and decoupled weight decay, with its generator replaced by class Gaussians from the exact class statistics, and FedDF with logit adjustment. Removing the coupled decay and adding logit adjustment recovers **[abl/cifar10/GeFL-F + LA/gain]** points of GeFL-F's deficit on CIFAR-10-LT; FedECS-LT adds **[abl/cifar10/rest]** points on top.

Components of FedECS-LT.. Table 14 removes one component of FedECS-LT at a time. [Write once the component ablation has been run (GUIDE.md, section 4, experiment E7): which components matter most, with and without the exact readout.]

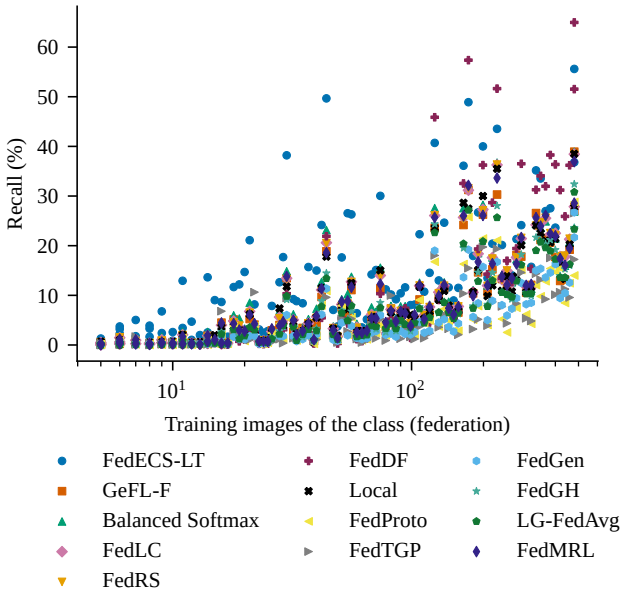


Figure 5: Recall of every class against its number of training images in the federation (CIFAR-100-LT, imbalance ratio 100, 10 clients; each point is one class, mean over seeds 3–5).

Robustness.. Table 15 varies the label skew, the number of clients and the imbalance ratio. [Write once the sweeps have been run (GUIDE.md, section 4, experiment E8).]

7.6. Representations and generated features

Figure 8 shows the test embeddings of GeFL-F and FedECS-LT on the same images. The embeddings of FedECS-LT separate the classes better: the leave-one-out 10-nearest-neighbour accuracy of the embeddings, which does not depend on the map, is 40.9% for FedECS-LT, 37.7% for GeFL-F and 38.9% for FedDF (mean over the ten architectures and seeds 3–5). It is higher for FedECS-LT in 29 of the 30 architecture–seed pairs against GeFL-F and in 24 against FedDF. Cluster sizes and the distances between clusters in such maps carry no meaning [63]; only the grouping does. Figure 9 overlays real and generated features.

7.7. Calibration, fairness across clients and cost

The expected calibration error of FedECS-LT is 37.8 ± 4.0 on CIFAR-10-LT against 48.0 ± 2.2 for GeFL-F and 35.9 ± 1.9 for FedDF, the strongest

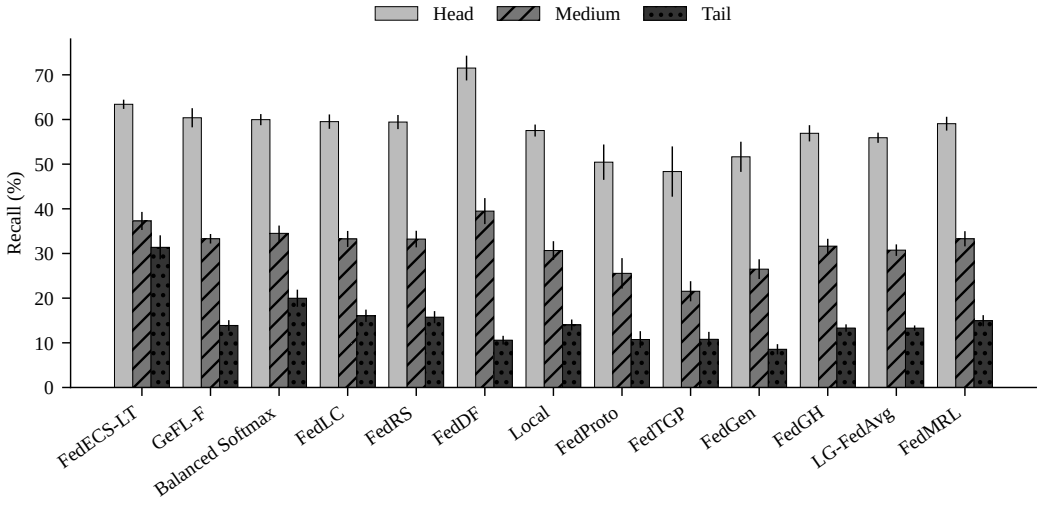


Figure 6: Recall of the head (30% most frequent), medium and tail (40% least frequent) classes, per method (CIFAR-10-LT, imbalance ratio 100, 10 clients; mean ± one standard deviation over seeds 3–5).

other method; the weakest architecture of FedECS-LT reaches 38.8 ± 1.9 balanced accuracy (Appendix C). Table 16 reports the cost on CIFAR-10-LT. FedECS-LT takes 1.13 times the wall-clock time of GeFL-F and 1.30 times its communication.

8. Discussion and limitations

What the gains consist of.. A large part of what any method of the GeFL-F family loses under a long tail is in its last layer, and an exact readout recovers it for every member of the family (Table 9). The difference between FedECS-LT and the strongest baseline is therefore smaller with the readout than without it; we report both. The comparison with the readout is closer to a comparison of the bodies, but the readout is the same linear classifier for every body, so it does not measure every property of a representation.

Limitations..

- The extractor is small and frozen, as in GeFL-F, and shared by all clients, so the clients' architectures differ only in their heads; we did not study pre-trained or larger extractors, or clients with different backbones.

Table 11: Balanced accuracy (%), heads as trained, on datasets with natural class imbalance (training sets capped at 20,000 images); mean ± standard deviation over seeds 3–5 (a value without ± is from one seed). 10 clients with ten different head architectures, Dirichlet $\alpha = 0.5$.

Method	DermaMNIST	OrganAMNIST	BloodMNIST	PathMNIST	EMNIST (ByClass)	FER-2013
FedECS-LT	**36.8 ± 0.8**	**83.7 ± 2.5**	**84.2 ± 2.0**	**61.9 ± 3.2**	**62.1**	**30.9 ± 1.5**
GeFL-F	27.3 ± 0.3	76.9 ± 3.5	78.5 ± 2.2	55.2 ± 2.3	49.8	27.5 ± 0.2
Balanced Softmax	28.6 ± 0.1	68.1 ± 4.8	69.9 ± 3.8	54.6 ± 2.2	45.6	26.6 ± 0.4
FedLC	26.0 ± 0.7	63.9 ± 4.4	66.5 ± 3.9	51.9 ± 2.6	42.1	25.7 ± 0.2
FedRS	26.0 ± 0.6	63.5 ± 4.4	66.0 ± 3.9	52.6 ± 1.4	41.4	25.7 ± 0.3
FedDF	20.5 ± 1.4	64.7 ± 6.1	70.6 ± 7.5	48.2 ± 1.2	47.2	25.7 ± 0.3
Local	25.3 ± 0.5	63.5 ± 4.2	63.6 ± 3.6	49.3 ± 3.4	41.4	25.2 ± 0.5
FedProto	25.5 ± 1.5	58.6 ± 4.1	65.4 ± 4.5	54.0 ± 0.8	39.4	20.8 ± 0.5
FedTGP	24.7 ± 1.0	50.7 ± 3.9	59.5 ± 2.4	49.8 ± 1.3	34.8	20.6 ± 0.7
FedGen	26.0 ± 0.6	60.1 ± 5.1	64.1 ± 3.2	51.7 ± 0.5	32.2	24.4 ± 0.5
FedGH	27.1 ± 0.2	61.8 ± 4.2	65.9 ± 3.6	53.8 ± 1.2	38.7	24.7 ± 0.4
LG-FedAvg	26.8 ± 0.4	62.6 ± 4.3	65.9 ± 3.2	53.3 ± 1.2	38.5	24.7 ± 0.6
FedMRL	27.1 ± 0.3	64.2 ± 4.3	67.2 ± 3.5	53.7 ± 1.0	42.1	25.9 ± 0.5

- The splits are pure label skew (Section 3); feature and concept shift between clients are not tested.
- Three seeds per setting give fragile intervals and paired tests; the tests are reported with Holm's correction, and differences of about a point should be read with that in mind.
- In the main setting each architecture has one client ($K = 10$); settings with several clients per architecture are covered only by Table 15.
- Secure aggregation hides each client's contribution from an honest-but-curious server, but the released federation-level sums are not differentially private: for classes with few samples they describe those samples closely. FedECS-LT should not be called privacy-preserving beyond that protocol guarantee.
- Some settings were run with fewer than three seeds; their tables say so and carry no interval.
- FedECS-LT costs more than GeFL-F (Table 16): its extra time is mostly in the longer synthetic phase of Stage III, and its extra communication mostly in the training of the generator, which with its projection of the class means is larger than GeFL-F's. We did not compare the methods at matched computation or communication; the ablation

Table 12: Balanced accuracy (%) with one ResNet-8 shared by all clients (FedLTLib protocol: 20 clients, 40% per round, 200 rounds, Dirichlet $\alpha = 0.5$); mean ± standard deviation over the seeds that came back (seeds 3–5; a value without ± is from one seed). Baselines in the authors' LongTailLib code on the same client splits, evaluated after the last round. Fed-GraB: accuracy of its own trained classifier as logged by the library.

Method	CIFAR-10	CIFAR-100
FedECS-LT	61.3 ± 1.8	34.1
FedLC	58.3 ± 1.9	32.8 ± 0.1
FedAvg	61.0 ± 1.8	33.2 ± 0.3
FedProx	54.4 ± 1.6	31.2 ± 0.4
MOON	59.9 ± 1.9	33.4 ± 0.2
CCVR	68.0 ± 1.2	35.8 ± 0.1
CReFF	56.8 ± 2.3	28.1 ± 0.0
FedETF	65.3 ± 4.3	32.8 ± 0.2
Fed-GraB	49.4 ± 2.4	27.8 ± 0.1
FedLoGe	66.7 ± 2.9	39.4
FedYoYo	**78.1 ± 0.9**	**42.6 ± 0.5**
RUCR	54.0 ± 1.6	32.9 ± 0.2

with one synthetic epoch shows how much FedECS-LT depends on the longer phase, not what the other methods would reach with it.

- The gate threshold F_0 and the other settings of Table 3 were chosen on earlier runs of the same benchmarks, partly with the same seeds and client splits and on the same test sets; they were fixed before these runs and not swept, and the baselines were not tuned (Section 6.10).

- With one model shared by all clients, the strongest long-tail methods (FedYoYo, FedLoGe, CCVR and FedETF on CIFAR-10-LT) are better (Table 12); FedECS-LT is designed for clients with different models and has no advantage when their models can be averaged.

- The absolute accuracy of the main setting is low. GeFL's authors note that GeFL-F is less effective on CIFAR-10 because of the lower performance of its feature extractor and target networks there [8]. In GeFL's own setting (a balanced training set divided evenly among 10 clients) our earlier implementation of GeFL-F reaches 56.8% on CIFAR-10 (59.3% with GeFL's best-round measure; GeFL's Fig. 4 shows about 55.9%) but only 12.5% on CIFAR-100,

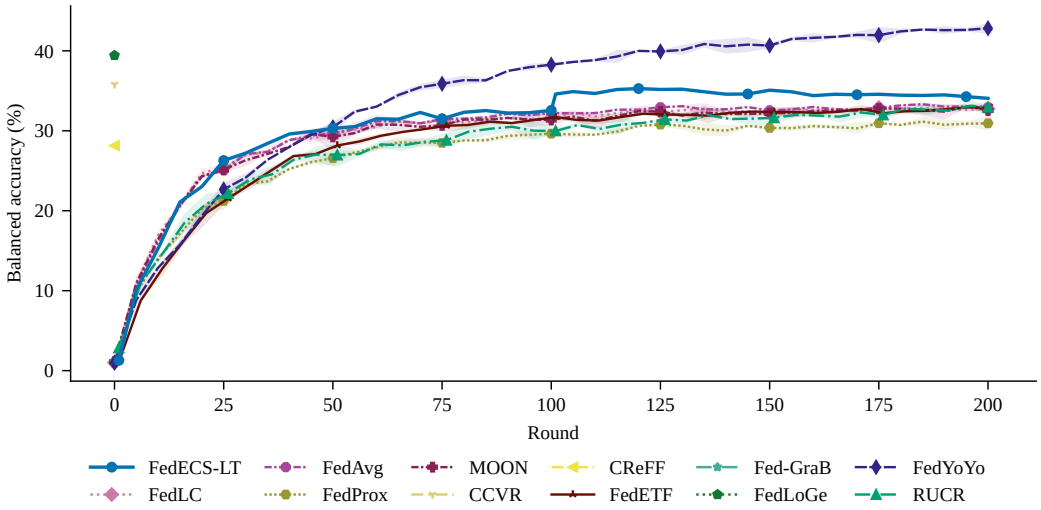


Figure 7: Test balanced accuracy over the communication rounds with one shared ResNet-8 (CIFAR-100-LT, imbalance ratio 100, 20 clients, 40% per round; mean over seeds 3–5). For FedECS-LT the rounds are those of the extractor and then of the head.

so on CIFAR-100 the level is set by the design and the data budget, and on CIFAR-10 the long tail, together with the smaller training set it leaves, costs most of the difference, and the same architectures, trained with cross-entropy on all the training data pooled (extractor and heads, no generator, no long-tail correction), reach **[main/cifar10/Centralised (pooled data)/bal]** on CIFAR-10-LT and **[main/cifar100/Centralised (pooled data)/bal]** on CIFAR-100-LT (**[main/cifar10/Centralised (pooled data)/er_bal]** and **[main/cifar100/Centralised (pooled data)/er_bal]** with the exact readout). These numbers are not comparable with centralised long-tail results for ResNet-32 and larger networks.

9. Conclusion

Generator-based model-heterogeneous federated learning loses rare classes in three places: the optimiser's treatment of clients that lack a class, a last layer fitted to one client's prior, and synthetic features that do not carry their label. Each can be described precisely, and each can be repaired with class statistics that secure aggregation computes up to its quantisation. FedECS-LT, which applies these repairs together, improves

Table 13: Variants of GeFL-F: balanced accuracy (%), heads as trained; mean ± standard deviation over seeds 3–5 (imbalance ratio 100, 10 clients, Dirichlet $\alpha = 0.5$).

Method	CIFAR-10-LT	CIFAR-100-LT
FedECS-LT	–	–
GeFL-F	–	–
GeFL-F w/o generator	–	–
GeFL-F + LA	–	–
GeFL-F w/ Gaussian generator + LA	–	–
FedDF	–	–
FedDF + LA	–	–

Placeholder: table slot `abl_bal` (GUIDE.md)

Table 14: Components of FedECS-LT: balanced accuracy (%) as trained and with the exact readout (ER) when one component is removed; mean ± standard deviation over seeds 3–5 (imbalance ratio 100, 10 clients).

	CIFAR-10-LT		CIFAR-100-LT	
Variant	as trained	ER	as trained	ER
FedECS-LT (full)	–	–	–	–
w/o mean anchor (class rows, CCWA)	–	–	–	–
w/o CCWA (flat averaging)	–	–	–	–
w/o moment correction	–	–	–	–
w/o consensus soft labels	–	–	–	–
w/o logit adjustment	–	–	–	–
$T_s = 1$ instead of 10	–	–	–	–

Placeholder: table slot `abl_components` (GUIDE.md)

balanced accuracy when clients have different models, on CIFAR-10-LT mostly through the recall of the tail classes; with one model shared by all clients, the strongest long-tail methods remain better. Its code, logs and every figure's data are public.

CRediT authorship contribution statement

[Author 1]: Conceptualization, Methodology, Software, Investigation, Writing – original draft. [Author 2]: Supervision, Writing – review & editing. [Use the CRediT roles that apply to each author.]

Table 15: Balanced accuracy (%) of FedECS-LT and the strongest baselines under other settings (CIFAR-10-LT unless stated); mean ± standard deviation over seeds 3–5.

	Dirichlet α		Clients K		Imbalance ratio	
Method	0.1	1.0	50	100	50	10
FedECS-LT	–	–	–	–	–	–
[strongest baseline 1]	–	–	–	–	–	–
[strongest baseline 2]	–	–	–	–	–	–
GeFL-F	–	–	–	–	–	–

Placeholder: table slot `robust` (GUIDE.md)

Table 16: Cost on CIFAR-10-LT (imbalance ratio 100, 10 clients): communication per client over the whole training, wall-clock time of a run on a T4 GPU (in most notebooks shared with a second run), forward MFLOPs per image and training GFLOPs per round; mean over seeds 3–5.

Method	Comm./client (MB)	Time (s)	MFLOPs/img	Train. GFLOPs/round
FedECS-LT	11,278 ± 113	7,910 ± 2,253	**2.45**	1,424 ± 112
GeFL-F	8,659	7,025 ± 446	**2.45**	579 ± 42
Balanced Softmax	60.8	542 ± 99	**2.45**	**485 ± 34**
FedLC	60.8	501 ± 1.2e+02	**2.45**	**485 ± 34**
FedRS	60.8	**417 ± 1.3e+02**	**2.45**	**485 ± 34**
FedDF	8,659	5,050 ± 2,736	**2.45**	579 ± 42
Local	-	536 ± 98	-	-
FedProto	18.8	4,025 ± 109	2.45	485 ± 34
FedTGP	18.8	4,121 ± 218	2.45	485 ± 34
FedGen	128	4,443 ± 1,033	2.45	485 ± 34
FedGH	18.8	3,149 ± 103	2.45	485 ± 34
LG-FedAvg	18.8	2,890 ± 212	2.45	485 ± 34
FedMRL	**17.1**	4,545 ± 804	3.84	871 ± 34

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

[Funding: name the grant and agency, or write: This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.]

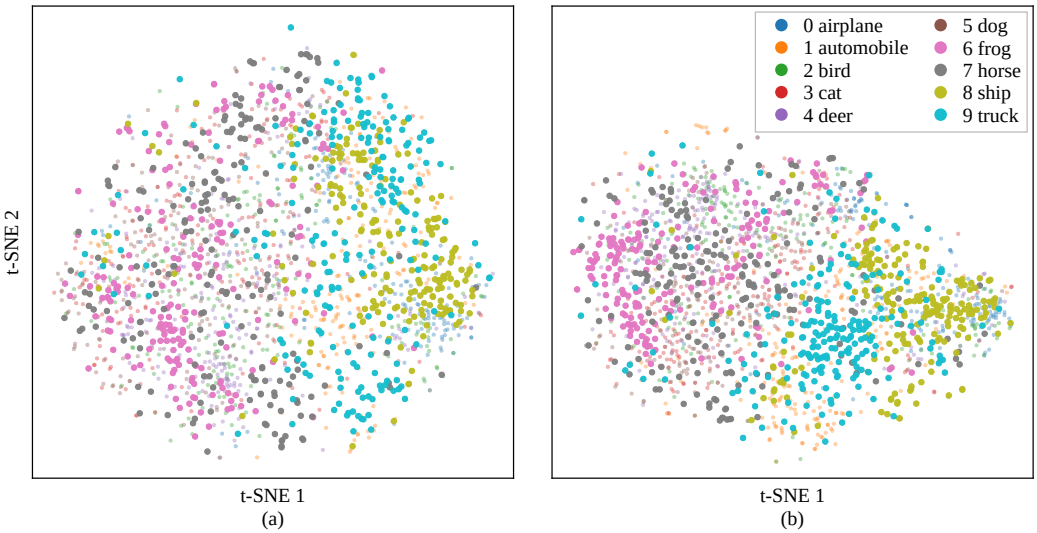


Figure 8: t-SNE [64] of the test embeddings of (a) GeFL-F and (b) FedECS-LT on the same 2,000 test images of CIFAR-10-LT (one architecture, seed 3; PCA to 50 dimensions, PCA initialisation, perplexity 30 [65]). Tail classes are drawn larger. Cluster sizes and distances between clusters are not interpretable.

Data and code availability

The code of FedECS-LT and of every compared method as run here, the notebooks, every run's logs and metrics and the data of every figure are available at [repository URL, e.g. a GitHub release with a DOI from Zenodo]. All datasets used are public (Section 6.3).

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used [name of the tool] in order to [e.g. write and test parts of the experiment code, draft and edit passages of the text]. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Acknowledgments

[Acknowledgments: computing resources, colleagues who helped. Or delete.]

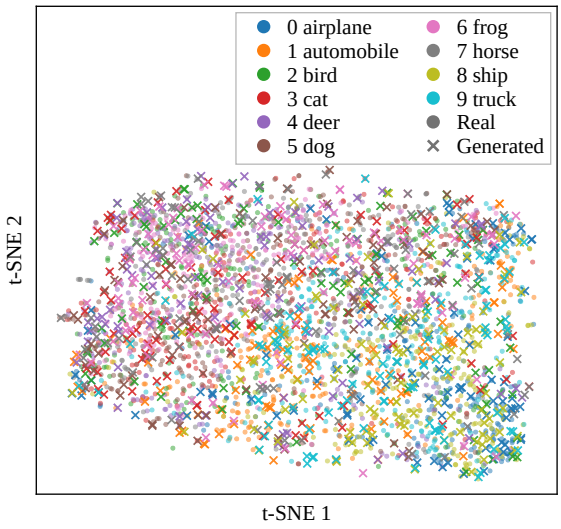


Figure 9: Real validation features (circles) and features generated by FedECS-LT (crosses) on CIFAR-10-LT, embedded together by t-SNE (seed 3; same settings as Fig. 8). Cluster sizes and distances between clusters are not interpretable.

References

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, B. Agüera y Arcas, Communication-efficient learning of deep networks from decentralized data, in: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS), Vol. 54 of PMLR, 2017, pp. 1273–1282.

[2] P. Kairouz, H. B. McMahan, et al., Advances and open problems in federated learning, Foundations and Trends in Machine Learning 14 (1–2) (2021) 1–210.

[3] Y. Tan, G. Long, L. Liu, T. Zhou, Q. Lu, J. Jiang, C. Zhang, FedProto: Federated prototype learning across heterogeneous clients, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 36, 2022, pp. 8432–8440.

[4] J. Zhang, Y. Liu, Y. Hua, J. Cao, FedTGP: Trainable global prototypes with adaptive-margin-enhanced contrastive learning for data and model heterogeneity in federated learning, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 38, 2024.

[5] Z. Zhu, J. Hong, J. Zhou, Data-free knowledge distillation for heterogeneous federated learning, in: Proceedings of the 38th International Conference on Machine Learning (ICML), Vol. 139 of PMLR, 2021.

[6] L. Yi, G. Wang, X. Liu, Z. Shi, H. Yu, FedGH: Heterogeneous federated learning with generalized global header, in: Proceedings of the 31st ACM International Conference on Multimedia (MM), 2023.

[7] L. Yi, H. Yu, C. Ren, G. Wang, X. Liu, X. Li, Federated model heterogeneous matryoshka representation learning, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 37, 2024.

[8] H. Kang, S. Cha, J. Kang, GeFL: Model-agnostic federated learning with generative models, IEEE Transactions on Mobile Computing 25 (4) (2026) 4493–4505, arXiv:2412.18460. doi:10.1109/TMC.2025.3592483.

[9] X. Shang, Y. Lu, G. Huang, H. Wang, Federated learning on heterogeneous and long-tailed data via classifier re-training with federated features, in: Proceedings of the 31st International Joint Conference on Artificial Intelligence (IJCAI), 2022.

[10] Z. Xiao, Z. Chen, S. Liu, H. Wang, Y. Feng, J. Hao, J. T. Zhou, J. Wu, H. H. Yang, Z. Liu, Fed-GraB: Federated long-tailed learning with self-adjusting gradient balancer, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 36, 2023.

[11] Z. Xiao, Z. Chen, L. Liu, Y. Feng, J. Wu, W. Liu, J. T. Zhou, H. H. Yang, Z. Liu, FedLoGe: Joint local and generic federated learning under long-tailed data, in: International Conference on Learning Representations (ICLR), 2024.

[12] S. Yan, Z. Li, C. Wu, M. Pang, Y. Lu, Y. Yan, H. Wang, You are your own best teacher: Achieving centralized-level performance in federated learning under heterogeneous and long-tailed data, in: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2025, pp. 2750–2759.

[13] W. Huang, Y. Liu, M. Ye, J. Chen, B. Du, Federated learning with long-tailed data via representation unification and classifier rectification, IEEE Transactions on Information Forensics and Security 19 (2024).

[14] D. P. Kingma, J. Ba, Adam: A method for stochastic optimization, in: International Conference on Learning Representations (ICLR), 2015.

[15] A. K. Menon, S. Jayasumana, A. S. Rawat, H. Jain, A. Veit, S. Kumar, Long-tail learning via logit adjustment, in: International Conference on Learning Representations (ICLR), 2021.

[16] J. Zhang, Z. Li, B. Li, J. Xu, S. Wu, S. Ding, C. Wu, Federated learning with label distribution skew via logits calibration, in: Proceedings of the 39th International Conference on Machine Learning (ICML), Vol. 162 of PMLR, 2022.

[17] J. Ren, C. Yu, S. Sheng, X. Ma, H. Zhao, S. Yi, H. Li, Balanced meta-softmax for long-tailed visual recognition, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 33, 2020, pp. 4175–4186.

[18] X.-C. Li, D.-C. Zhan, FedRS: Federated learning with restricted softmax for label distribution non-IID data, in: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), 2021.

[19] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, K. Seth, Practical secure aggregation for privacy-preserving machine learning, in: Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS), 2017, pp. 1175–1191.

[20] J. H. Bell, K. A. Bonawitz, A. Gascón, T. Lepoint, M. Raykova, Secure single-server aggregation with (poly)logarithmic overhead, in: Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS), 2020, pp. 1253–1269.

[21] M. Luo, F. Chen, D. Hu, Y. Zhang, J. Liang, J. Feng, No fear of heterogeneity: Classifier calibration for federated learning with non-IID data, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 34, 2021, pp. 5972–5984.

[22] E. Fanì, R. Camoriano, B. Caputo, M. Ciccone, Accelerating heterogeneous federated learning with closed-form classifiers, in: Proceedings of the 41st International Conference on Machine Learning (ICML), Vol. 235 of PMLR, 2024.

[23] G. Legate, N. Bernier, L. Page-Caccia, E. Oyallon, E. Belilovsky, Guiding the last layer in federated learning with pre-trained models, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 36, 2023.

[24] D. Goswami, S. Magistri, K. Wang, B. Twardowski, A. D. Bagdanov, J. van de Weijer, Covariances for free: Exploiting mean distributions for training-free federated learning, in: Advances in Neural Information Processing Systems (NeurIPS), 2025, arXiv:2412.14326.

[25] Z. Niu, H. Dong, A. K. Qin, Bridging generalization gap of heterogeneous federated clients using generative models, in: International Conference on Learning Representations (ICLR), 2026.

[26] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, J. Fernandez-Marques, Y. Gao, L. Sani, K. H. Li, T. Parcollet, P. P. B. de Gusmão, N. D. Lane, Flower: A friendly federated learning research framework, arXiv preprint arXiv:2007.14390 (2020).

[27] J. Zhang, X. Wu, Y. Zhou, X. Sun, Q. Cai, Y. Liu, Y. Hua, Z. Zheng, J. Cao, Q. Yang, HtFLlib: A comprehensive heterogeneous federated learning library and benchmark, in: Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), 2025.

[28] C. Lin, J. Wang, FedLTLib: A comprehensive benchmark for federated long-tail learning, arXiv preprint arXiv:2609.15625 (2026).

[29] D. Li, J. Wang, FedMD: Heterogenous federated learning via model distillation, arXiv preprint arXiv:1910.03581 (2019).

[30] T. Lin, L. Kong, S. U. Stich, M. Jaggi, Ensemble distillation for robust model fusion in federated learning, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 33, 2020, pp. 2351–2363.

[31] P. P. Liang, T. Liu, L. Ziyin, N. B. Allen, R. P. Auerbach, D. Brent, R. Salakhutdinov, L.-P. Morency, Think locally, act globally: Federated learning with local and global representations, arXiv preprint arXiv:2001.01523 (2020).

[32] L. Zhang, L. Shen, L. Ding, D. Tao, L.-Y. Duan, Fine-tuning global model via data-free knowledge distillation for non-IID federated learning, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022.

[33] R. He, K. Tong, D. Fang, H. Sun, Z. Zeng, H. Li, T. Chen, H. Zhuang, AFL: A single-round analytic approach for federated learning with pre-trained models, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2025, arXiv:2405.16240.

[34] Z. Li, X. Shang, R. He, T. Lin, C. Wu, No fear of classifier biases: Neural collapse inspired federated learning with synthetic and fixed classifier, in: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2023.

[35] H.-Y. Chen, W.-L. Chao, On bridging generic and personalized federated learning for image classification, in: International Conference on Learning Representations (ICLR), 2022.

[36] Z. Chen, Z. Wu, Y. Ji, Decoupled federated learning on long-tailed and non-IID data with feature statistics, arXiv:2403.08364 (2024).

[37] F. X. Yu, A. S. Rawat, A. K. Menon, S. Kumar, Federated learning with only positive labels, in: Proceedings of the 37th International Conference on Machine Learning (ICML), Vol. 119 of PMLR, 2020.

[38] Y. Cui, M. Jia, T.-Y. Lin, Y. Song, S. Belongie, Class-balanced loss based on effective number of samples, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2019, pp. 9268–9277.

[39] B. Kang, S. Xie, M. Rohrbach, Z. Yan, A. Gordo, J. Feng, Y. Kalantidis, Decoupling representation and classifier for long-tailed recognition, in: International Conference on Learning Representations (ICLR), 2020.

[40] I. Loshchilov, F. Hutter, Decoupled weight decay regularization, in: International Conference on Learning Representations (ICLR), 2019.

[41] M. Shahbazi, M. Danelljan, D. P. Paudel, L. Van Gool, Collapse by conditioning: Training class-conditional GANs with limited data, in: International Conference on Learning Representations (ICLR), 2022.

[42] S. Ravuri, O. Vinyals, Classification accuracy score for conditional generative models, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 32, 2019.

[43] R. A. Fisher, The use of multiple measurements in taxonomic problems, Annals of Eugenics 7 (2) (1936) 179–188. doi:10.1111/j.1469-1809.1936.tb02137.x.

[44] F. Harder, K. Adamczewski, M. Park, DP-MERF: Differentially private mean embeddings with random features for practical privacy-preserving data generation, in: Proceedings of the 24th International Conference on Artificial Intelligence and Statistics (AISTATS), Vol. 130 of PMLR, 2021.

[45] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, V. Smith, Federated optimization in heterogeneous networks, in: Proceedings of Machine Learning and Systems (MLSys), Vol. 2, 2020, pp. 429–450.

[46] Q. Li, B. He, D. Song, Model-contrastive federated learning, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2021, pp. 10713–10722.

[47] J. Zhang, Y. Hua, H. Wang, T. Song, Z. Xue, R. Ma, H. Guan, FedALA: Adaptive local aggregation for personalized federated learning, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 37, 2023, pp. 11237–11244. doi:10.1609/aaai.v37i9.26330.

[48] A. Afonin, S. P. Karimireddy, Towards model agnostic federated learning using knowledge distillation, in: International Conference on Learning Representations (ICLR), 2022, arXiv:2110.15210.

[49] T.-M. H. Hsu, H. Qi, M. Brown, Measuring the effects of non-identical data distribution for federated visual classification, arXiv preprint arXiv:1909.06335 (2019).

[50] K. Sohn, H. Lee, X. Yan, Learning structured output representation using deep conditional generative models, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 28, 2015.

[51] C. Fang, H. He, Q. Long, W. J. Su, Exploring deep neural networks via layer-peeled model: Minority collapse in imbalanced training, Proceedings of the National Academy of Sciences 118 (43) (2021) e2103091118.

[52] D. Soudry, E. Hoffer, M. S. Nacson, S. Gunasekar, N. Srebro, The implicit bias of gradient descent on separable data, Journal of Machine Learning Research 19 (70) (2018) 1–57.

[53] A. K. Menon, A. S. Rawat, S. Reddi, S. Kim, S. Kumar, A statistical perspective on distillation, in: Proceedings of the 38th International Conference on Machine Learning (ICML), Vol. 139 of PMLR, 2021.

[54] A. Krizhevsky, Learning multiple layers of features from tiny images, Tech. rep., University of Toronto (2009).

[55] J. Yang, R. Shi, D. Wei, Z. Liu, L. Zhao, B. Ke, H. Pfister, B. Ni, MedMNIST v2 – a large-scale lightweight benchmark for 2D and 3D biomedical image classification, Scientific Data 10 (2023) 41.

[56] G. Cohen, S. Afshar, J. Tapson, A. van Schaik, EMNIST: Extending MNIST to handwritten letters, in: International Joint Conference on Neural Networks (IJCNN), 2017, pp. 2921–2926.

[57] I. J. Goodfellow, D. Erhan, P. L. Carrier, A. Courville, M. Mirza, B. Hamner, W. Cukierski, Y. Tang, D. Thaler, D.-H. Lee, et al., Challenges in representation learning: A report on three machine learning contests, in: International Conference on Neural Information Processing (ICONIP), Vol. 8228 of Lecture Notes in Computer Science, 2013, pp. 117–124.

[58] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778.

[59] J. Gorodkin, Comparing two K-category assignments by a K-category correlation coefficient, Computational Biology and Chemistry 28 (5–6) (2004) 367–374.

[60] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger, On calibration of modern neural networks, in: Proceedings of the 34th International Conference on Machine Learning (ICML), Vol. 70 of PMLR, 2017, pp. 1321–1330.

[61] M. Kull, M. Perello Nieto, M. Kängsepp, T. Silva Filho, H. Song, P. Flach, Beyond temperature scaling: Obtaining well-calibrated multi-class probabilities with Dirichlet calibration, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 32, 2019.

[62] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.

[63] M. Wattenberg, F. Viégas, I. Johnson, How to use t-SNE effectively, Distill (2016). doi:10.23915/distill.00002.

[64] L. van der Maaten, G. Hinton, Visualizing data using t-SNE, Journal of Machine Learning Research 9 (2008) 2579–2605.

[65] D. Kobak, P. Berens, The art of using t-SNE for single-cell transcriptomics, Nature Communications 10 (2019) 5416.

[66] M. Gelbrich, On a formula for the L2 Wasserstein metric between measures on Euclidean and Hilbert spaces, Mathematische Nachrichten 147 (1) (1990) 185–203.

Appendix A. Proofs

Proposition 1.. By the triangle inequality, $\|\frac{1}{K}\sum_k r_{k,c}\| \le \frac{1}{K}\sum_k \|r_{k,c}\| = \|r_c\| + \frac{1}{K}\big(\sum_{k\in H_c}\Delta_k + \sum_{k\notin H_c}\Delta_k\big)$, and the two sums are $m_c\bar{G}_c$ and $-(K-m_c)\bar{D}_c$. Equality holds if and only if all vectors are non-negative multiples of one another. □

Lemma 2.. A non-holder's batches contain no sample of class c, so its data loss does not depend on r_c and the gradient of the regularised loss is λr_c. Adam's update $\eta\hat{m}/(\sqrt{\hat{v}}+\epsilon)$ is unchanged when the whole gradient sequence is multiplied by $a > 0$, up to the ϵ term, so the step does not depend on λ while $\lambda|r_{c,j}| \gg \epsilon$. A client keeps its Adam state across rounds while its parameters are reset to the global ones, so a single step can exceed η. □

The step ratio (Eq. (1)), a heuristic.. Per coordinate an Adam step is $\eta\psi$ with $\psi = \hat{m}/\sqrt{\hat{v}}$. For a gradient of constant sign ψ tends to that sign; in general $|\psi| \le (1-\beta_1)/\sqrt{1-\beta_2}$, about 3.2 for the default β_1, β_2 [14]. A non-holder sees a decay gradient of constant sign, so $\psi \approx \operatorname{sign}(r_{c,j})$ on each of its q_k steps. Assume that a holder moves the row only on the $q_k f_{kc}$ steps whose batch

contains class c, with $|\psi| \le 1$, and neglect its other steps. Under flat averaging the drift per round is $\Delta r_{c,j} = \frac{\eta}{K}\big[\psi_h \sum_{k\in H_c} q_k f_{kc} - \mathrm{sign}(r_{c,j}) \sum_{k\notin H_c} q_k\big]$, which points toward zero for every coordinate and every holder direction when $\kappa_c < 1$. The approximation neglects momentum carried between sparse holder steps and the drift of $\hat{v}$. □

Proposition 3.. (1) $f + a_k = Wu(h) + (b + a_k)$, and $b \mapsto b + a_k$ is a translation with identity Jacobian, so the gradients agree at corresponding points; induct over the steps. (2) $\alpha f_c = \tilde{w}_c^\top u + \tilde{b}_c$ with $\tilde{w}_c = \alpha w_c$, $\tilde{b}_c = \alpha b_c$; the gradient in $\tilde{w}_c$ is $1/\alpha$ times that in w_c, so a step of size η on w_c is a step of $\eta\alpha^2$ on $\tilde{w}_c$, and the body sees the same logits. □

Proposition 4.. Addition is associative and commutative. In particular the class-balanced linear discriminant of Eq. (8) is computed exactly by one round of secure aggregation. □

Proposition 5.. $\frac{1}{C}\sum_c(\mathrm{LSE}(f) - f_c) = \mathrm{LSE}(f) - \bar{f}$ and $\mathrm{KL}(u\|\mathrm{softmax} f) = -\log C + \mathrm{LSE}(f) - \bar{f}$; expanding LSE around constant logits gives the quadratic term. □

Proposition 6 (alternation, quadratic losses)**.** *Let the synthetic and real losses be quadratic with Hessians $H_s, H_r \succeq 0$ and minimisers θ_s, θ_r, and let $P_s = (I - \eta H_s)^{S_s}$, $P_r = (I - \eta H_r)^{S_r}$ for S_s synthetic and S_r real steps per round. Let $0 \le \eta\lambda \le 1$ for every eigenvalue λ of H_s and H_r. If $\|P_r P_s\| < 1$, the rounds converge to $\theta^\star = \theta_r + \Omega(\theta_s - \theta_r)$ with $\Omega = (I - P_r P_s)^{-1} P_r (I - P_s)$. For commuting Hessians the synthetic weight along eigendirection i is $\omega_i = \rho_{r,i}(1 - \rho_{s,i})/(1 - \rho_{r,i}\rho_{s,i}) \le \rho_{r,i}$, which equals one along directions that the real loss leaves free and the synthetic loss does not.*

Proof. One round is the affine map $\theta \mapsto P_r P_s \theta + P_r(I - P_s)\theta_s + (I - P_r)\theta_r$, a contraction when $\|P_r P_s\| < 1$; its fixed point has the stated form. With commuting Hessians all matrices are diagonal in a common eigenbasis. □

Proposition 7 (what a mean anchor can express)**.** *Let the class-c sampler be $X_c = \mathrm{ReLU}(\mu_c + g(Z, A\mu_c))$ with Z shared across classes and g L-Lipschitz in its second argument. Then $W_1(G_c, G_{c'}) \le (1 + L\|A\|_2)\|\mu_c - \mu_{c'}\|$.*

Proof. Couple both classes through the same Z; ReLU is 1-Lipschitz. □

In units of the root-mean-square within-class spread, two class means are on average $\sqrt{2FC/(C-1)}$ apart (over distinct pairs). When F is small the anchor carries little class information, and the factor $1 + L\|A\|_2$ is not controlled; this is why the gate of Section 5.2 falls back to class rows.

Proposition 8 (least-displacement moment correction)**.** *Among all maps T with $\mathbb{E}\,T(X) = \mu_c$ and $\mathbb{E}\|T(X) - \mu_c\|^2 = V_c$, where X has mean $\tilde{\mu}_c$ and spread $\tilde{V}_c$, the map $T^\star(x) = \mu_c + \sqrt{V_c/\tilde{V}_c}\,(x - \tilde{\mu}_c)$ minimises $\mathbb{E}\|T(X) - X\|^2$. With exact covariances instead of spreads the minimiser is the linear map attaining the Gelbrich bound [66].*

Proof. With $Y = T(X)$, $\mathbb{E}\|Y - X\|^2 = V_c + \tilde{V}_c + \|\mu_c - \tilde{\mu}_c\|^2 - 2\mathbb{E}\langle Y - \mu_c, X - \tilde{\mu}_c\rangle$, and the Cauchy–Schwarz inequality bounds the last expectation by $\sqrt{V_c\tilde{V}_c}$, with equality if and only if $Y - \mu_c = \sqrt{V_c/\tilde{V}_c}(X - \tilde{\mu}_c)$. The deployed map adds a final ReLU and clips the scale, so it matches the moments only approximately. □

Appendix B. Experimental details

Models.. The extractor, the ten head architectures (CNN-1 to CNN-10) and GeFL-F's conditional VAE follow GeFL-F [8]; heads are trained by SGD with learning rate 0.1. Rounds of the three stages: 20/100/50 on MNIST and Fashion-MNIST, 50/100/100 on SVHN and 50/200/100 on CIFAR-10 and CIFAR-100; with natural imbalance, 20/100/50 on OrganAMNIST and EMNIST and 50/100/100 on DermaMNIST, BloodMNIST, PathMNIST, GTSRB and FER-2013.

Reproduction check.. Before the comparison, the Flower implementation was checked against our earlier, single-process implementation (which follows GeFL's code) on MNIST-LT and CIFAR-10-LT with seed 3. The client splits were identical (every client's class counts), every client training step matched weight for weight in unit tests, and the balanced accuracy of GeFL-F and FedECS-LT was within the larger of two points and three seed standard deviations of the earlier runs. The earlier CIFAR-10 reference of FedECS-LT was an older configuration, so its tolerance was three points.

Head architectures.. CNN-1 to CNN-10 have convolutional blocks of widths (16), (16, 32), (20, 40), (10, 20), (16, 32, 64), (20, 40, 80), (10, 20, 40), (16, 32, 64, 128), (20, 40, 80, 100) and (10, 20, 40, 80). Each block is a 3×3 convolution, batch normalisation (Fashion-MNIST, SVHN and the naturally imbalanced datasets only), ReLU and 2×2 max pooling, followed by a linear last layer.

FedDF.. After each head round the server distils every architecture for 20 SGD steps (batch 128, learning rate 0.05, no momentum) on fresh generated features with uniformly drawn labels; the target is the mean softmax of the ten architectures, and batch-normalisation statistics are kept. FedDF's original method distils on unlabelled public data instead.

GeFL's own setting.. In GeFL's setting (balanced training data divided evenly among 10 clients, 3 seeds, our earlier implementation), GeFL-F reaches 56.8% on CIFAR-10 after the last round and 59.3% with GeFL's measure, against 55.9% read from GeFL's Fig. 4 [8], which reports each model's best round. An earlier configuration of FedECS-LT (one synthetic epoch per round and an older head training) reaches 57.8% and 60.0%. On CIFAR-100, which GeFL does not report, GeFL-F reaches 12.5% and that configuration 18.0%.

Evaluation.. Models are evaluated every five rounds on the test set before and after aggregation; the final model is the one after the last round (no selection on the test or the validation set). Methods with personal heads (LG-FedAvg, FedGH, FedGen and the other HtFLlib methods) are scored with each client's head after its last local training; HtFLlib scores LG-FedAvg, FedGH and FedGen right after the server's update instead. FedProto and FedTGP classify by the distance to class prototypes; their probabilities for the calibration metrics are a softmax of the negative mean squared distances.

HtFLlib ports.. Every client keeps the frozen Stage-I extractor and starts from its Stage-I head body, followed by a parameter-free pooling to 512 dimensions and a linear layer, as in HtFLlib's models. Training uses HtFLlib's run-script settings (SGD, learning rate 0.01, batch 10) with 5 local epochs per round for the rounds of the head stage; personal models persist across rounds. FedGen's label counts are the federation's class counts.

Appendix C. Further results

Per-dataset tables of every metric are generated from the same runs (Tables C.17 and C.18).

Table C.17: CIFAR-10-LT (imbalance ratio 100, 10 clients, Dirichlet $\alpha = 0.5$): main metrics (%), heads as trained; mean ± standard deviation over seeds 3–5. Tail: 40% least frequent classes; G-mean with recalls floored at 1%.

Method	Bal. acc.	Macro-F1	Precision	MCC	G-mean	Tail rec.	Worst rec.	AUC-PR
FedECS-LT	**42.8 ± 2.0**	**40.9 ± 2.3**	**47.9 ± 1.2**	**37.3 ± 2.0**	**37.5 ± 2.7**	**31.4 ± 2.7**	**22.5 ± 2.2**	**45.5 ± 2.4**
GeFL-F	33.7 ± 1.2	26.9 ± 1.4	38.3 ± 3.3	28.2 ± 1.4	15.7 ± 1.8	13.9 ± 1.2	4.8 ± 1.1	34.0 ± 2.2
Balanced Softmax	36.3 ± 1.2	30.4 ± 1.2	39.5 ± 2.7	30.9 ± 1.3	19.1 ± 1.6	20.0 ± 1.9	9.8 ± 1.2	36.3 ± 2.0
FedLC	34.3 ± 1.1	27.4 ± 0.9	40.2 ± 2.8	29.0 ± 1.2	15.2 ± 0.9	16.1 ± 1.4	6.2 ± 0.2	35.4 ± 1.8
FedRS	34.1 ± 1.1	27.2 ± 0.9	40.3 ± 2.9	28.9 ± 1.2	15.0 ± 0.9	15.7 ± 1.4	6.0 ± 0.3	35.3 ± 1.8
FedDF	37.5 ± 1.2	30.6 ± 1.4	43.7 ± 5.1	32.1 ± 1.3	18.3 ± 1.9	10.6 ± 1.0	1.6 ± 0.8	41.5 ± 1.9
Local	32.1 ± 0.7	25.0 ± 0.7	37.5 ± 2.3	26.8 ± 0.8	13.3 ± 0.6	14.0 ± 1.2	5.0 ± 0.7	33.9 ± 1.5
FedProto	27.1 ± 0.8	21.0 ± 0.4	27.6 ± 0.8	20.4 ± 0.8	12.3 ± 0.5	10.7 ± 1.9	3.8 ± 3.4	26.3 ± 0.6
FedTGP	25.3 ± 1.2	18.5 ± 1.3	26.1 ± 1.2	19.1 ± 1.0	9.9 ± 1.0	10.8 ± 1.7	4.5 ± 4.3	26.9 ± 0.6
FedGen	26.9 ± 2.0	19.7 ± 1.7	27.3 ± 1.7	20.5 ± 2.2	10.0 ± 1.2	8.6 ± 1.1	2.3 ± 0.3	22.4 ± 2.0
FedGH	31.9 ± 1.1	24.6 ± 1.0	35.7 ± 3.4	26.4 ± 1.3	12.8 ± 0.7	13.3 ± 0.8	3.5 ± 0.3	31.6 ± 1.7
LG-FedAvg	31.3 ± 0.8	24.4 ± 0.5	33.7 ± 1.1	25.6 ± 0.8	13.3 ± 0.4	13.3 ± 0.6	4.3 ± 0.5	29.7 ± 0.6
FedMRL	33.7 ± 1.0	26.7 ± 0.9	38.6 ± 3.7	28.3 ± 1.1	14.9 ± 1.0	14.9 ± 1.2	5.1 ± 1.0	32.7 ± 1.5

Table C.18: CIFAR-100-LT: main metrics (%), as Table C.17.

Method	Bal. acc.	Macro-F1	Precision	MCC	G-mean	Tail rec.	Worst rec.	AUC-PR
FedECS-LT	**14.3 ± 0.6**	**12.6 ± 0.5**	**15.7 ± 0.6**	**13.5 ± 0.6**	**7.9 ± 0.4**	**6.4 ± 0.3**	**0.2 ± 0.2**	**11.1 ± 0.5**
GeFL-F	7.7 ± 0.4	4.7 ± 0.3	6.0 ± 0.4	6.9 ± 0.4	2.9 ± 0.1	1.7 ± 0.1	0.0 ± 0.0	4.4 ± 0.2
Balanced Softmax	9.0 ± 0.3	5.8 ± 0.3	7.3 ± 0.4	8.2 ± 0.3	3.3 ± 0.1	2.2 ± 0.1	0.0 ± 0.0	5.1 ± 0.3
FedLC	8.5 ± 0.3	5.3 ± 0.2	7.7 ± 0.5	7.8 ± 0.3	3.0 ± 0.1	1.7 ± 0.1	0.0 ± 0.0	5.0 ± 0.3
FedRS	8.5 ± 0.3	5.3 ± 0.2	7.9 ± 0.5	7.7 ± 0.3	2.9 ± 0.1	1.7 ± 0.1	0.0 ± 0.0	5.0 ± 0.3
FedDF	10.0 ± 0.5	6.1 ± 0.3	9.7 ± 0.9	9.3 ± 0.5	3.3 ± 0.1	0.8 ± 0.1	0.0 ± 0.0	7.0 ± 0.4
Local	7.9 ± 0.8	4.9 ± 0.6	7.3 ± 0.7	7.2 ± 0.8	2.8 ± 0.2	1.6 ± 0.2	0.0 ± 0.0	4.8 ± 0.5
FedProto	4.8 ± 0.3	3.0 ± 0.1	3.9 ± 0.3	3.9 ± 0.3	2.2 ± 0.1	0.9 ± 0.0	0.0 ± 0.0	2.9 ± 0.1
FedTGP	4.5 ± 0.6	2.4 ± 0.4	3.4 ± 0.5	3.7 ± 0.6	1.9 ± 0.2	1.2 ± 0.5	0.0 ± 0.0	2.8 ± 0.6
FedGen	4.5 ± 1.3	2.6 ± 0.9	3.1 ± 0.8	3.7 ± 1.2	2.0 ± 0.3	0.7 ± 0.3	0.0 ± 0.0	2.2 ± 0.6
FedGH	6.5 ± 0.1	3.9 ± 0.2	5.0 ± 0.5	5.7 ± 0.1	2.5 ± 0.1	1.1 ± 0.1	0.0 ± 0.0	3.5 ± 0.1
LG-FedAvg	6.3 ± 0.1	3.7 ± 0.1	4.8 ± 0.2	5.4 ± 0.1	2.4 ± 0.0	1.0 ± 0.0	0.0 ± 0.0	3.3 ± 0.1
FedMRL	8.3 ± 0.2	5.0 ± 0.2	6.9 ± 0.1	7.5 ± 0.2	2.9 ± 0.1	1.7 ± 0.1	0.0 ± 0.0	4.4 ± 0.1

Table C.19: GeFL's measure on the long-tailed benchmarks: each client model's best test accuracy (%) at GeFL's evaluation rounds of the head stage (the first, every tenth and the last), averaged over the ten models, as in GeFL's code; mean ± standard deviation over seeds 3–5. The main tables report the last round instead.

Method	CIFAR-10	CIFAR-100
FedECS-LT	**46.1 ± 2.4**	**15.2 ± 0.5**
GeFL-F	34.7 ± 1.3	8.4 ± 0.4
Balanced Softmax	38.3 ± 1.8	9.2 ± 0.4
FedLC	34.6 ± 1.3	8.6 ± 0.3
FedRS	34.4 ± 1.3	8.5 ± 0.3
FedDF	35.6 ± 1.2	9.8 ± 0.5
Local	32.1 ± 0.7	7.9 ± 0.8
FedProto	32.8 ± 1.3	7.6 ± 0.3
FedTGP	32.3 ± 1.4	7.6 ± 0.2
FedGen	32.1 ± 1.3	7.7 ± 0.2
FedGH	33.3 ± 1.4	7.8 ± 0.2
LG-FedAvg	33.0 ± 1.3	7.7 ± 0.2
FedMRL	33.9 ± 1.1	8.3 ± 0.2

Table C.20: Accuracy (%) on each client's own class mix (the test accuracy re-weighted to the class frequencies of the model's own clients), heads as trained, on the long-tailed benchmarks; mean ± standard deviation over seeds 3–5.

Method	CIFAR-10	CIFAR-100
FedECS-LT	68.6 ± 1.9	31.5 ± 0.3
GeFL-F	73.0 ± 2.3	31.1 ± 0.6
Balanced Softmax	72.8 ± 2.3	33.1 ± 0.7
FedLC	74.2 ± 2.1	34.7 ± 0.7
FedRS	**74.2 ± 2.1**	**34.8 ± 0.8**
FedDF	71.5 ± 2.2	32.9 ± 1.1
Local	72.7 ± 2.6	32.3 ± 2.4
FedProto	61.3 ± 1.1	17.0 ± 1.3
FedTGP	62.7 ± 3.4	21.1 ± 1.3
FedGen	67.0 ± 2.7	22.0 ± 4.2
FedGH	72.6 ± 2.3	29.4 ± 0.8
LG-FedAvg	71.0 ± 2.2	28.0 ± 0.4
FedMRL	73.5 ± 2.2	33.7 ± 0.7

Table C.21: Calibration on the long-tailed benchmarks: expected calibration error (%, 15 bins; lower is better), heads as trained; mean ± standard deviation over seeds 3–5.

Method	CIFAR-10	CIFAR-100
FedECS-LT	37.8 ± 4.0	41.4 ± 0.9
GeFL-F	48.0 ± 2.2	42.4 ± 0.7
Balanced Softmax	55.2 ± 0.9	74.3 ± 0.4
FedLC	58.1 ± 0.8	75.8 ± 0.4
FedRS	58.3 ± 0.8	76.0 ± 0.5
FedDF	35.9 ± 1.9	31.8 ± 1.2
Local	53.2 ± 3.8	69.0 ± 3.4
FedProto	17.1 ± 0.8	**3.8 ± 0.3**
FedTGP	**8.8 ± 2.9**	25.8 ± 13.3
FedGen	67.3 ± 1.7	73.9 ± 13.9
FedGH	57.3 ± 0.8	71.2 ± 0.3
LG-FedAvg	61.0 ± 0.3	77.2 ± 0.2
FedMRL	60.2 ± 0.9	77.5 ± 0.5

Table C.22: Fairness across clients on the long-tailed benchmarks: balanced accuracy (%) of the weakest head architecture, heads as trained; mean ± standard deviation over seeds 3–5.

Method	CIFAR-10	CIFAR-100
FedECS-LT	**38.8 ± 1.9**	**11.7 ± 0.9**
GeFL-F	27.5 ± 2.4	6.0 ± 0.8
Balanced Softmax	30.4 ± 0.9	7.1 ± 1.1
FedLC	28.1 ± 1.1	6.8 ± 0.9
FedRS	27.8 ± 1.1	6.8 ± 0.8
FedDF	32.2 ± 2.8	8.4 ± 1.5
Local	25.5 ± 2.6	5.1 ± 1.5
FedProto	18.7 ± 3.3	3.5 ± 0.2
FedTGP	16.8 ± 1.9	1.7 ± 0.7
FedGen	20.5 ± 1.6	3.1 ± 1.8
FedGH	25.6 ± 2.2	5.2 ± 0.5
LG-FedAvg	25.3 ± 1.1	4.9 ± 0.5
FedMRL	27.4 ± 0.9	7.1 ± 0.3